A Project Report

On

FINDLY – COMMUNITY LOST & FOUND PORTAL

Submitted by

RISHU GUPTA

In partial fulfillment for the award of the degree

of

BACHELOR OF SCIENCE

In

COMPUTER SCIENCE

Under the guidance

Of

PROF. T.SUJAH

THEEM COLLEGE OF ARTS, COMMERCE AND SCIENCE

(Affiliated to University of Mumbai)

BOISAR, MAHARASHTRA – 401501

Sem v

2026-2027

THEEM COLLEGE OF ART COMMERCE & SCIENCE

(Affiliated to University of Mumbai)

BOISAR-401501

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that **RISHU GUPTA** of **T.Y.BSC. CS** (Sem V), seat no. 15008 has satisfactorily completed the Project **Findly – Community Lost & Found Portal**, to be submitted in the partial fulfillment for the award of **Bachelor of Science** in **Computer Science** during the academic year **2026 – 2027.**

Internal Guide **Coordinator**

External Examiner

Date: **College Seal**

DECLARATION

I, **Mr. RISHU GUPTA**, student of **Theem College of Arts, Commerce and Science**, studying in the **Bachelor of Science in Computer Science** program, hereby declare that I have completed the **Community Engagement Project** titled **"DIGITAL LOST AND FOUND FOR PUBLIC SPACE"** during the academic year **2026–2027**.

This project has been developed under the guidance of my project guide. The work presented in this project report is based on my own efforts and understanding. The information and resources used during the development of this website have been properly acknowledged in the references section.
I further declare that this project has not been submitted to any other university or institution for the award of any degree or diploma.

Place: BOISAR
Date: _____
Student Name: Rishu Gupta
Class: TY BSC(CS)
Roll No.: 15008
Signature: _____

ACKNOWLEDGEMENT

We sincerely express our gratitude to our respected Project Guide for providing valuable guidance, suggestions, and continuous support throughout the development of **"Findly – Community Lost & Found Portal."**

We are thankful to the Head of the Department and all faculty members for their academic guidance, technical support, and encouragement. We also thank our college for providing the necessary infrastructure and resources for completing this project.

We are grateful to our classmates and friends for their valuable feedback and cooperation during development and testing. We also thank our parents and family for their constant motivation and support.

Finally, we thank everyone who directly or indirectly contributed to the successful completion of this project. This project has been a valuable learning experience that improved our technical, teamwork, and problem-solving skills.

ABSTRACT

Findly – Community Lost & Found Portal is a web-based application designed to help users report, search, and recover lost and found belongings in an organized way.

Users can register and post details of lost or found items such as mobile phones, wallets, ID cards, books, and bags. The system provides a centralized platform with **user authentication, search, database management, and item listing** features.

The project reduces dependence on notice boards and messaging groups by making lost-and-found information easily accessible. It also provides practical experience in **frontend and backend development, database integration, testing, and deployment**.

In the future, Findly can be enhanced with **automatic item matching, notifications, image recognition, location-based search, and an admin dashboard**.

TABLE OF CONTENTS

Sr. No.	Section	Title / Description	Page No.
		Cover Page	1
		Certificate	2
		Declaration	
		Acknowledgement	3
		Abstract	4
		Table of Contents	6
		List of Figures	8
1	Chapter 1	PROBLEM IDENTIFICATION & FEASIBILITY STUDY	-
	1.1	Identification of a Real-World Problem	9
	1.2	Problem Justification	9
	1.3	Scope Definition	9
	1.4	Stakeholder Identification	9
	1.5	Feasibility Study	9
	1.5.1	Technical Feasibility	9
	1.5.2	Economic Feasibility	9
	1.5.3	Operational Feasibility	9
2	Chapter 2	REQUIREMENT ENGINEERING	-
	2.1	Functional Requirements Specification	10
	2.2	Non-Functional Requirements	10
	2.3	Use-Case Analysis	10
	2.4	Requirement Prioritization	10
	2.5	Constraints and Assumptions	10
3	Chapter 3	SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) PLANNING	-
	3.1	Selection of SDLC Model	12
	3.2	Work Breakdown Structure (WBS)	12
	3.3	Project Timeline	13
	3.4	Gantt Chart	13
	3.5	Resource Planning	13
4	Chapter 4	SYSTEM MODELING USING UML	-
	4.1	Use Case Diagram	14
	4.2	Class Diagram	15
	4.3	Sequence Diagram	16
	4.4	Activity Diagram	17
	4.5	Entity Relationship (ER) Diagram	18
	4.6	Deployment Diagram	19
5	Chapter 5	SYSTEM ARCHITECTURE DESIGN	-
	5.1	Frontend Architecture	20
	5.2	Backend Architecture	20
	5.3	Database Schema Design	21
	5.4	API Structure	22
	5.5	Security Considerations	24

6	Chapter 6	CODING OF WEBSITE	-
	6.1	Frontend coding	25
	6.1.1	Login.ejs	25
	6.1.2	Register.ejs	25
	6.1.3	Form.ejs	26
	6.1.4	Index.ejs	26
	6.1.5	Show.ejs	28
	6.1.6	Main.ejs	29
	6.1.7	Admin.ejs	30
	6.1.8	Dashboard.ejs	31
	6.1.9	Error.ejs	32
	6.1.10	Home.ejs	32
	6.2	Backend coding	34
	6.2.1	Server.js	34
	6.2.2	Auth.js	38
	6.2.3	Claim.js	39
	6.2.4	Item.js	39
	6.2.5	user.js	39
	6.2.6	Package.json	39
7	Chapter 7	APPLICATION DEVELOPMENT	-
	7.1	Frontend Implementation	41
	7.2	Backend Implementation	42
	7.3	Database Integration	43
	7.4	Authentication and Validation	43
	7.5	Error Handling	43
8	Chapter 8	INTEGRATION & SYSTEM TESTING	-
	8.1	Unit Testing	44
	8.2	Black-Box Testing	44
	8.3	Integration Testing	44
	8.4	Test Case Preparation	44
	8.5	Bug Tracking	45
9	Chapter 9	APPLICATION DEPLOYMENT	-
	9.1	Cloud Deployment	46
	9.2	Local Hosting	46
	9.3	APK Build	46
	9.4	Server Configuration	46
	9.5	Version Control Using GitHub	47
10	Chapter 10	PERFORMANCE & SECURITY TESTING	-
	10.1	Basic Load Testing	48
	10.2	Input Validation Checks	48
	10.3	Security Validation	48
11	Chapter 11	FINAL DOCUMENTATION	-
	11.1	Technical Report	49
	11.2	User Manual	49
	11.3	Application Screenshots	49
	11.4	Source Code Documentation	51
12	Chapter 12	REFERENCE	52
13	Chapter 13	GLOSSARY	53
14	Chapter 14	APPENDICES	55

LIST OF FIGURES

Figure No.	Caption / Description	Page
Fig. 1	Use Case Diagram	-
Fig. 2	Class Diagram	-
Fig. 3	Sequence Diagram	-
Fig. 4	Activity Diagram	-
Fig. 5	Class Diagram	-
Fig. 6	Sequence Diagram	-
Fig. 7	Entity Relationship (ER) Diagram	-
Fig. 8	Deployment Diagram	-
Fig. 9	Database Schema Design	-
Fig. 10	Website Navigation	-
Fig. 11	Security Considerations	-
Fig. 12	Frontend coding	-
Fig. 13	Backend coding	-
Fig. 14	Findly Home Page	-
Fig. 15	Findly Sign-in Page	-
Fig. 16	Findly Account Creation Page	-
Fig. 17	Findly Dashboard	-

SCREEN SHOT OF OUR WEBSITE

Figure No.	Caption / Description	Page
Fig. 1	Home page	-
Fig. 2	Create account in Findly	-
Fig. 3	Sign in to Findly	-
Fig. 4	Main Dashboard	-

1. Problem Identification & Feasibility Study

1.1 Identification of a Real-World Problem

Loss of personal belongings such as **mobile phones, wallets, ID cards, keys, documents, and bags** is a common issue in colleges and communities. Existing methods like WhatsApp groups, social media, and notice boards are unorganized and lack a centralized search mechanism.

FINDLY addresses this problem by providing a centralized digital platform to connect people who have **lost** items with those who have **found** them.

1.2 Problem Justification

The absence of a structured lost-and-found system can result in:

- Difficulty locating reported items.
- Scattered and outdated information.
- Lack of proper ownership verification.
- Delayed communication between owners and finders.

FINDLY provides a **systematic, searchable, and secure approach** to improve the recovery process.

1.3 Scope Definition

FINDLY will provide:

- User registration and authentication.
- Lost and found item reporting.
- Image and item-detail uploads.
- Search and filtering facilities.
- Potential lost–found matching.
- Ownership claims and verification.
- Notifications and status tracking.
- Administrative management of users and reports.

1.4 Stakeholder Identification

The primary stakeholders are:

- **Users/Owners:** Report and search for lost belongings.
- **Finders:** Report items they have found.
- **Administrators:** Manage users, reports, claims, and system activities.
- **Community Organizations:** Use the platform to facilitate item recovery.

1.5 Feasibility Study

1.5.1 Technical Feasibility: FINDLY can be implemented using modern web technologies such as **HTML, CSS, JavaScript/React, Node.js/PHP, and MySQL/MongoDB**. The required development tools and technologies are readily available.

1.5.2 Economic Feasibility: The system can be developed using **open-source technologies and low-cost/free development tools**, making it suitable for a college-level project.

1.5.3 Operational Feasibility: The system will provide a **simple and user-friendly interface**, enabling users to report, search, match, and claim items without requiring technical expertise.

2. Requirement Engineering

Requirement Engineering defines what the FINDLY system should do, what users expect from it, and the conditions under which it should operate. It provides a clear foundation for designing and developing the system.

2.1 Functional Requirements Specification

Functional requirements describe the **features and operations** that FINDLY must provide.

- **User Registration & Login:** Users can create accounts and securely log in.
- **Lost Item Reporting:** Users can submit details of items they have lost.
- **Found Item Reporting:** Users can register items they have found.
- **Item Search:** Users can search for lost or found items using keywords and filters.
- **Item Details:** The system displays item description, category, location, date, and images.
- **Matching:** The system identifies potential matches between lost and found reports.
- **Claim Submission:** Users can submit a claim for an item they believe belongs to them.
- **Verification:** Claims can be reviewed before an item is marked as returned.
- **Notifications:** Users receive updates about potential matches and claim status.
- **Admin Management:** Administrators can manage users, reports, claims, and inappropriate content.

2.2 Non-Functional Requirements

Non-functional requirements define **how effectively and securely** the system should operate.

- **Performance:** Pages and searches should respond quickly.
- **Security:** User credentials and personal information must be protected.
- **Usability:** The interface should be simple and easy to understand.
- **Reliability:** The system should provide consistent results and minimize failures.
- **Scalability:** The system should support an increasing number of users and reports.
- **Availability:** The portal should be accessible whenever users need it.
- **Maintainability:** The software should be easy to update and maintain.

2.3 Use-Case Analysis

Use-case analysis identifies the interaction between **users and the FINDLY system**.

Main actors:

- User
- Finder
- Admin

Major Use Cases:

- Register/Login
- Report Lost Item
- Report Found Item
- Search Items
- View Item Details
- Submit Claim
- Receive Notifications
- Verify Claims
- Manage Users and Reports

2.4 Requirement Prioritization

Requirements are prioritized according to their importance.

Priority	Requirements
High	Login, Lost/Found Reporting, Search, Database
Medium	Matching, Claims, Notifications
Low	Advanced analytics, recommendations, additional features

The **high-priority features** are essential for the basic operation of FINDLY.

2.5 Constraints and Assumptions

Constraints:

- Requires an internet connection for online access.
- Users must provide reasonably accurate information.
- System resources and hosting capacity may be limited.
- Ownership verification depends on the information provided by users.

Assumptions:

- Users will provide genuine lost/found information.
- Users will use the claim process responsibly.
- Administrators will monitor reports and claims.
- Users have access to a smartphone or computer with a web browser.

3. Software Development Life Cycle (SDLC) Planning

SDLC Planning defines the **development approach, project activities, timeline, and resources** required to develop the FINDLY system in a structured manner.

3.1 Selection of SDLC Model

For FINDLY, the **Agile SDLC model** can be used because it supports incremental development and continuous improvement.

The project can be divided into small iterations:

Requirement Analysis → Design → Development → Testing → Feedback → Improvement

This approach allows features such as reporting, searching, matching, and claiming to be developed and tested progressively.

3.2 Work Breakdown Structure (WBS)

The project can be divided into the following major activities:

1. **Requirement Analysis**
   - Identify user requirements
   - Define functional requirements
   - Define non-functional requirements

2. **System Design**
   - Design database
   - Prepare UML diagrams
   - Design user interface

3. **Development**
   - Develop authentication
   - Develop lost/found reporting
   - Develop search and filtering
   - Develop matching and claim modules
   - Develop admin panel

4. **Testing**
   - Unit testing
   - Integration testing
   - System testing
   - User acceptance testing

5. **Deployment & Documentation**
   - Deploy the application
   - Prepare user documentation
   - Prepare project report

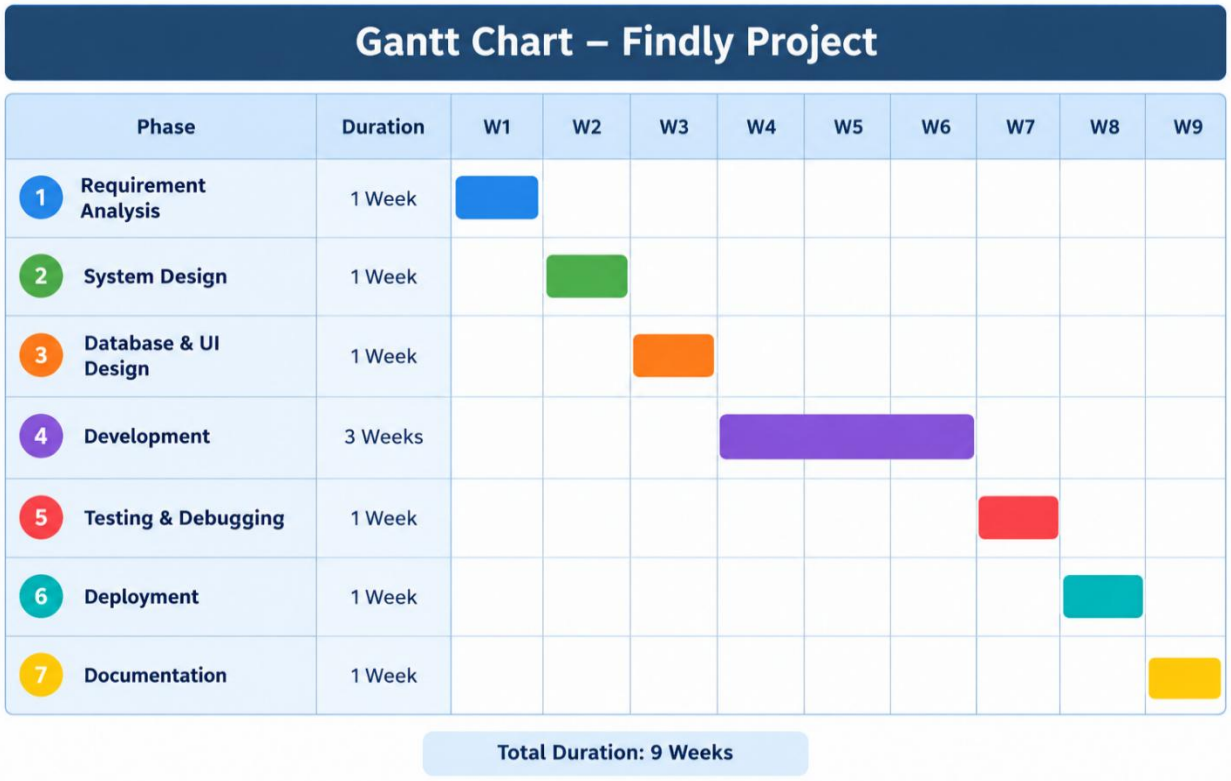
3.3 Project Timeline

A possible timeline for a mini-project is:

Phase	Duration
Requirement Analysis	1 Week
System Design	1 Week
Database & UI Design	1 Week
Development	3 Weeks
Testing & Debugging	1 Week
Deployment	1 Week
Documentation	1 Week

3.4 Gantt Chart

The project schedule can be represented using a **Gantt Chart**, showing the start date, duration, and completion status of each development activity.



3.5 Resource Planning

The resources required for FINDLY include:
Hardware: Computer/Laptop, internet connection, and sufficient storage.
Software: VS Code, web browser, Git/GitHub, database software, and required development frameworks.
Human Resources: Project developer/team members, project guide, tester, and administrator.

4. System Modelling Using UML

System Modelling represents the structure, behavior, and interaction of the FINDLY system using **UML (Unified Modelling Language) diagrams**. These diagrams make the system easier to understand before implementation.

4.1 Use Case Diagram

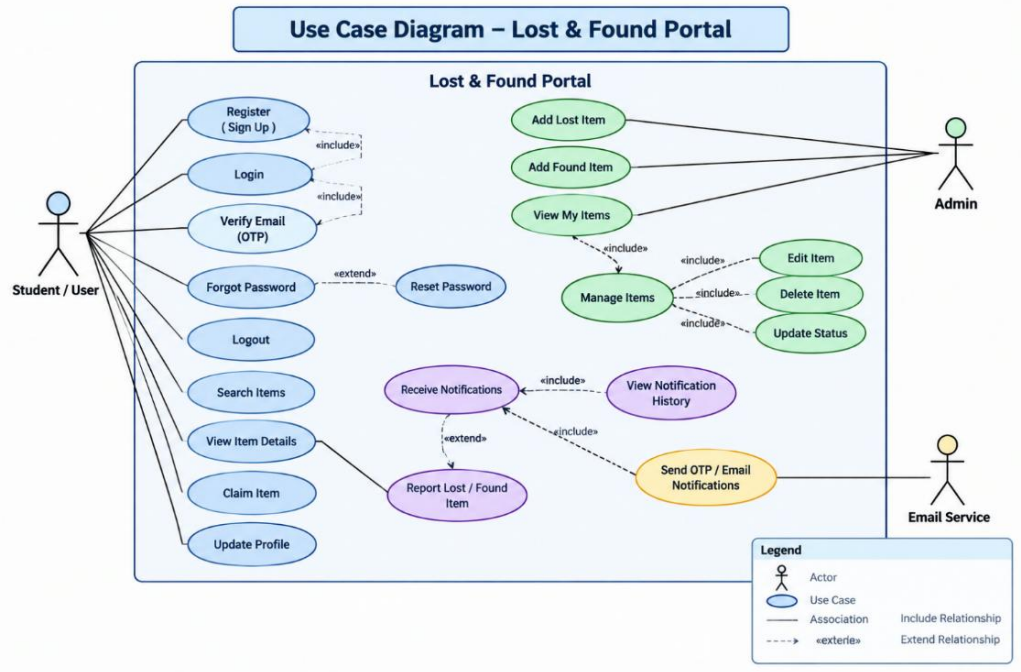
The Use Case Diagram shows how different users interact with FINDLY.

Main Actors:

- User
- Finder
- Admin

Major Use Cases:

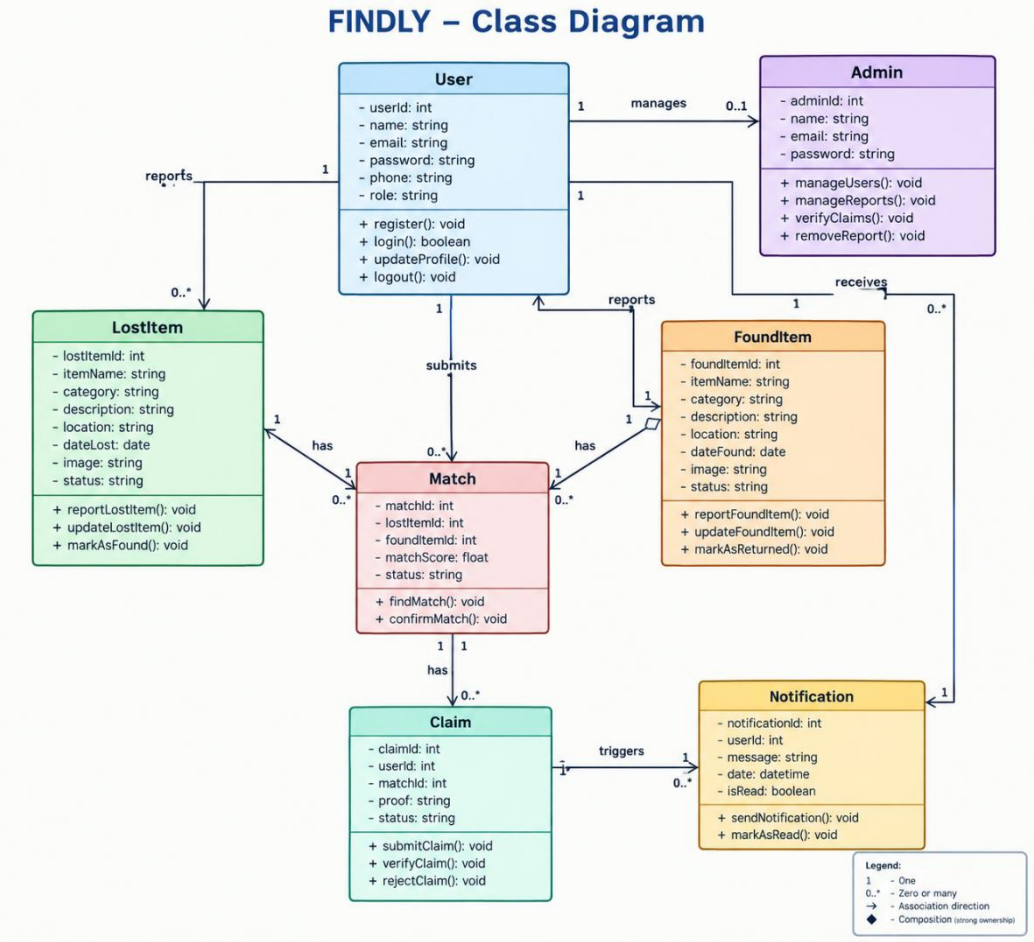
- Register/Login
- Report Lost Item
- Report Found Item
- Search Items
- View Item Details
- Submit Claim
- Receive Notifications
- Verify Claims
- Manage Users and Reports



4.2 Class Diagram

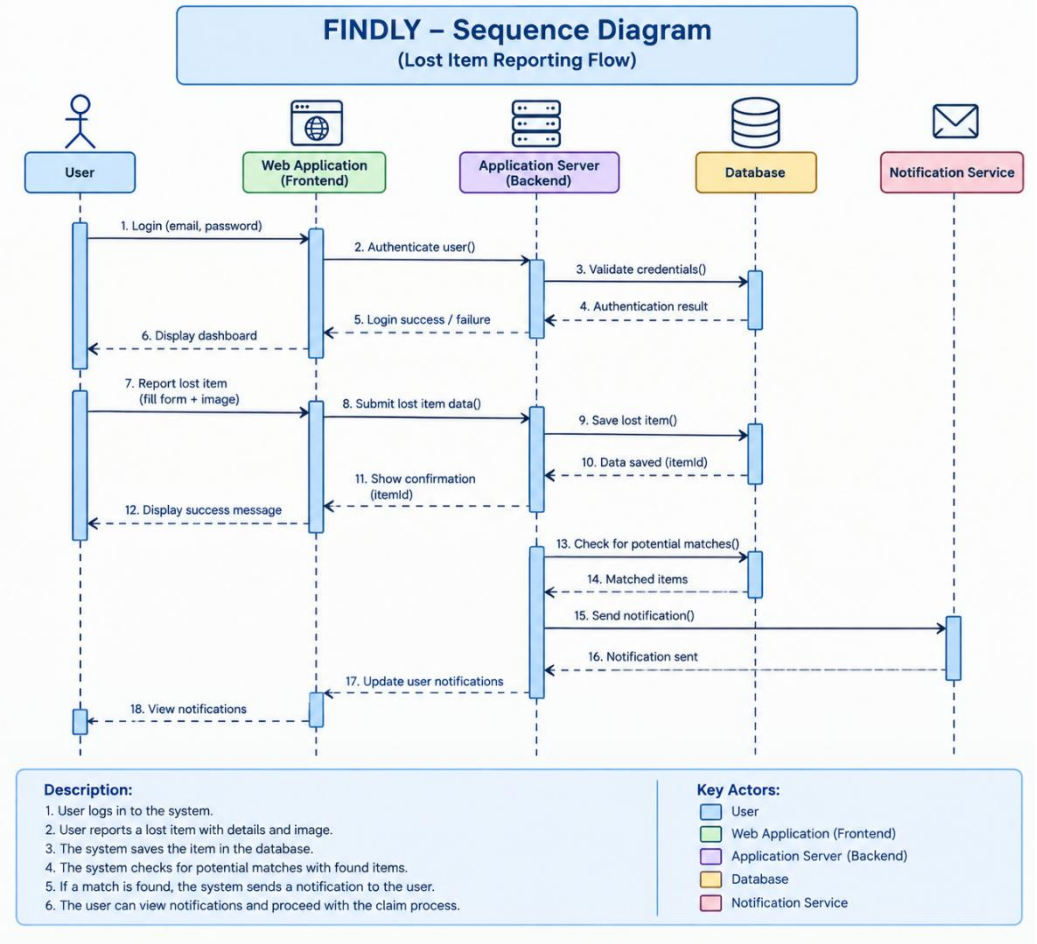
The Class Diagram represents the **main entities, attributes, and relationships** in FINDLY. Important classes include:

- **User**
- **Lost Item**
- **Found Item**
- **Match**
- **Claim**
- **Notification**
- **Admin**



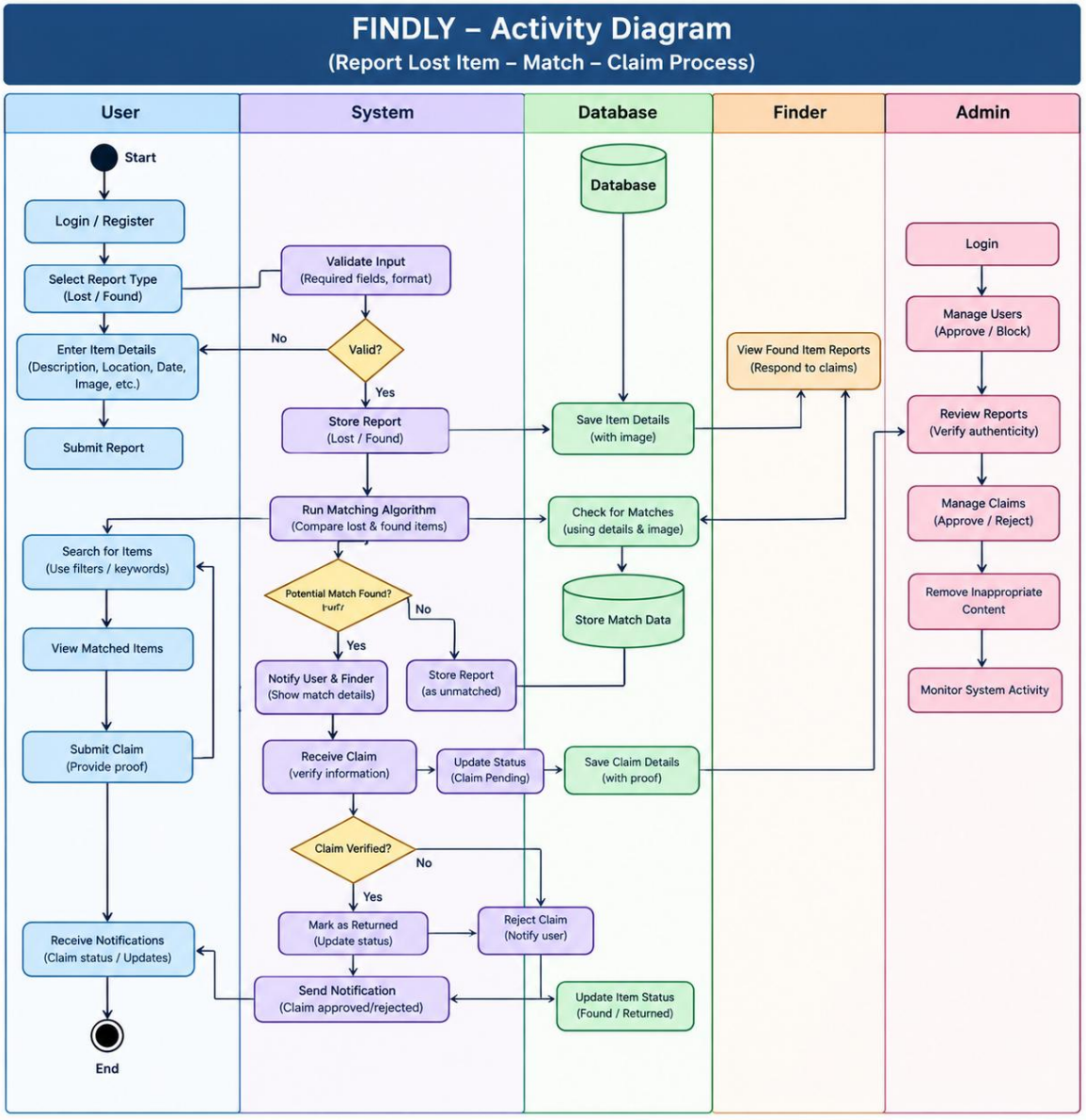
4.3 Sequence Diagram

The Sequence Diagram shows the **order of communication between users and the system**.



4.4 Activity Diagram

The Activity Diagram represents the **workflow of a particular process**.

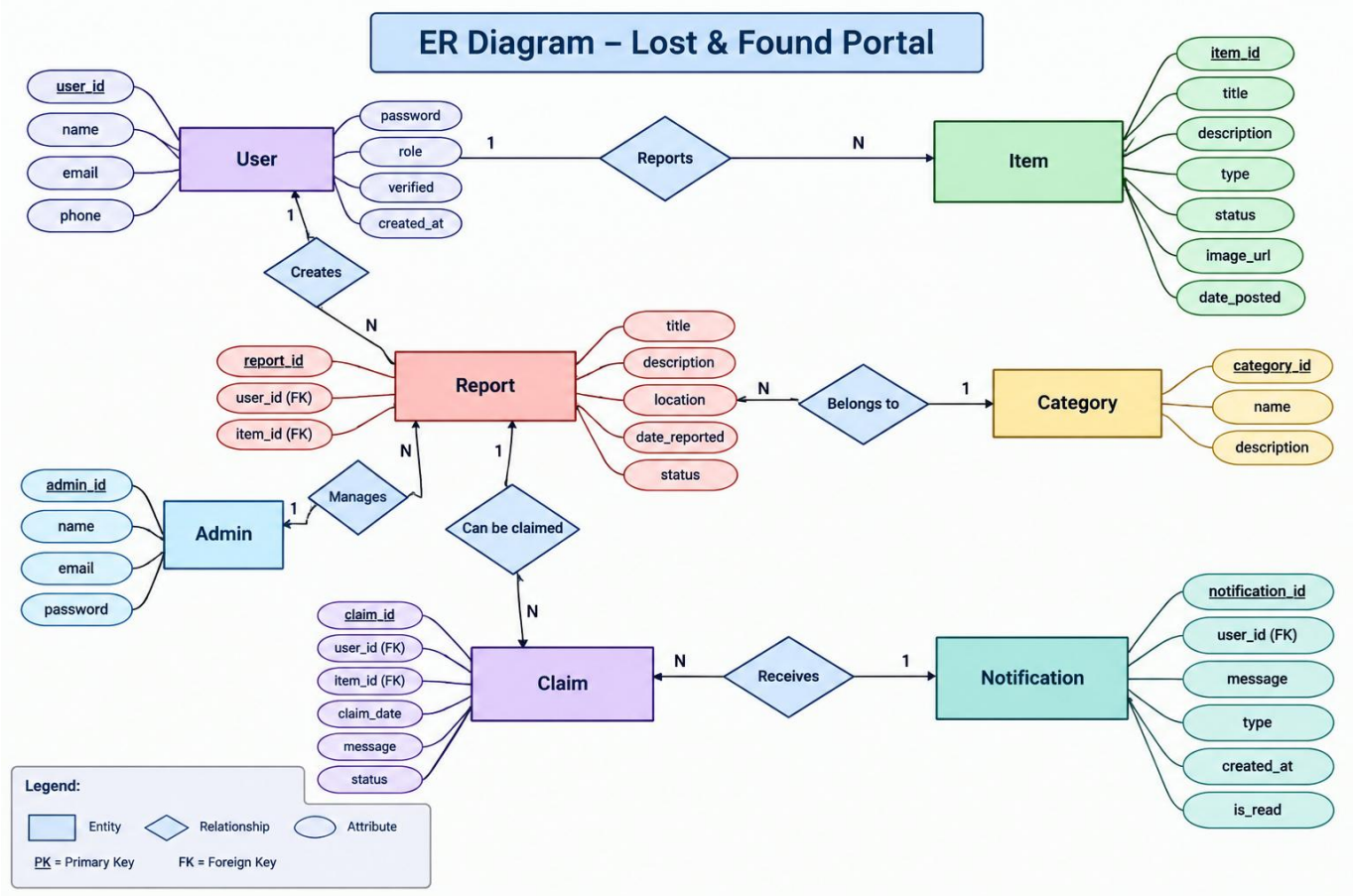


4.5 Entity Relationship (ER) Diagram

The ER Diagram represents the **database structure and relationships**.

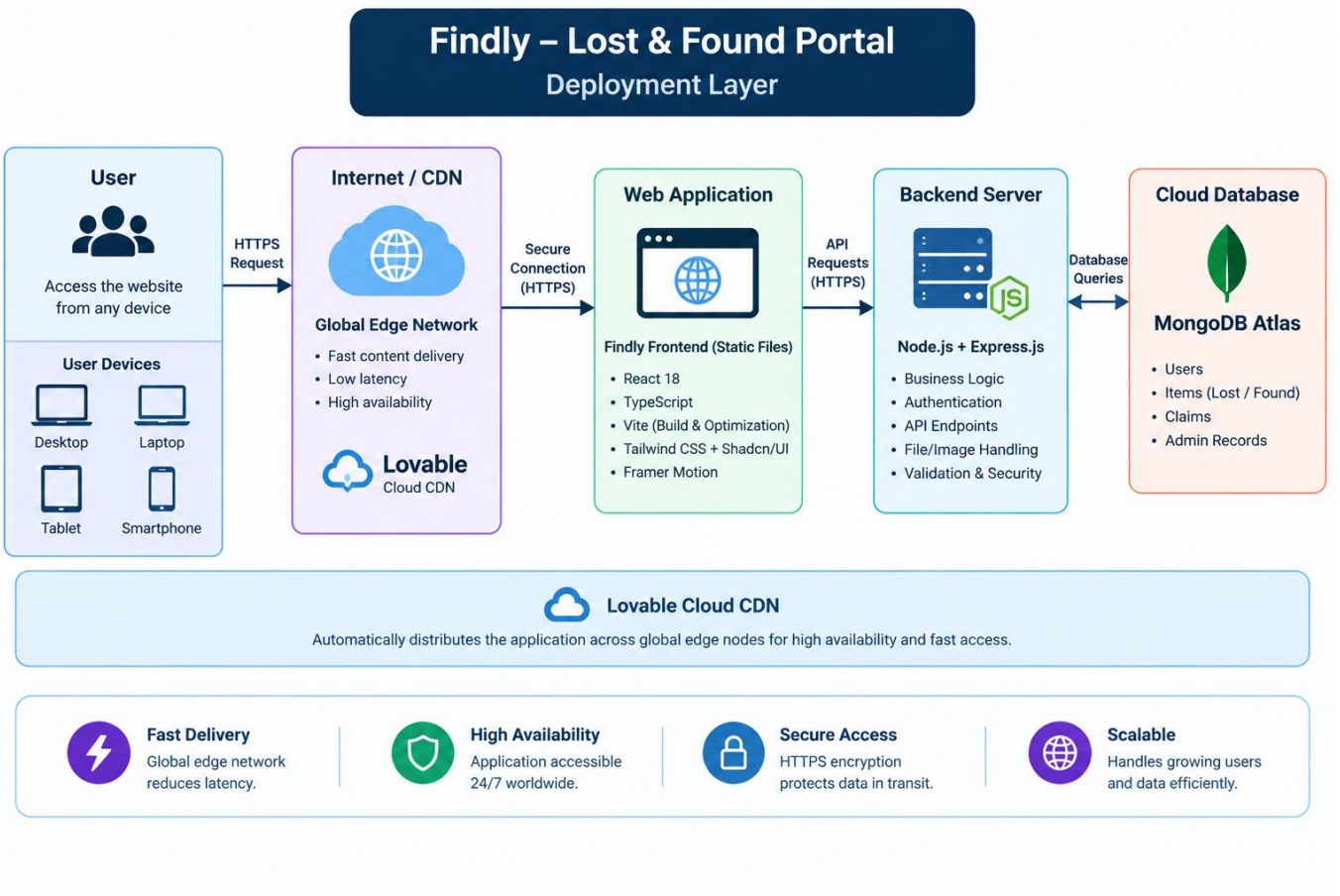
Main entities:

- User
- Lost Item
- Found Item
- Match
- Claim
- Notification



4.6 Deployment Diagram

The Deployment Diagram shows where the FINDLY application components are deployed.



It represents the physical arrangement of the system and communication between its components.

5. System Architecture Design

System Architecture Design defines the **overall technical structure of the FINDLY website** and explains how its major components interact.

5.1 Frontend Architecture

The frontend is the **user-facing component** of FINDLY.

Main Pages

- Home Page
- Registration
- Login
- User Dashboard
- Report Lost Item
- Report Found Item
- Search Items
- Item Details
- Claims
- Admin Dashboard

Technologies

- HTML
- CSS
- JavaScript
- React.js

The frontend is designed to provide a **responsive, accessible, and user-friendly interface**.

5.2 Backend Architecture

The backend manages the **business logic and application processing**.
It handles:

- User authentication
- Lost/found reports
- Search and filtering
- Matching
- Claim verification
- Notifications
- Administrative operations
- Database communication

The backend receives requests from the website, processes them, communicates with the database, and returns the required response.

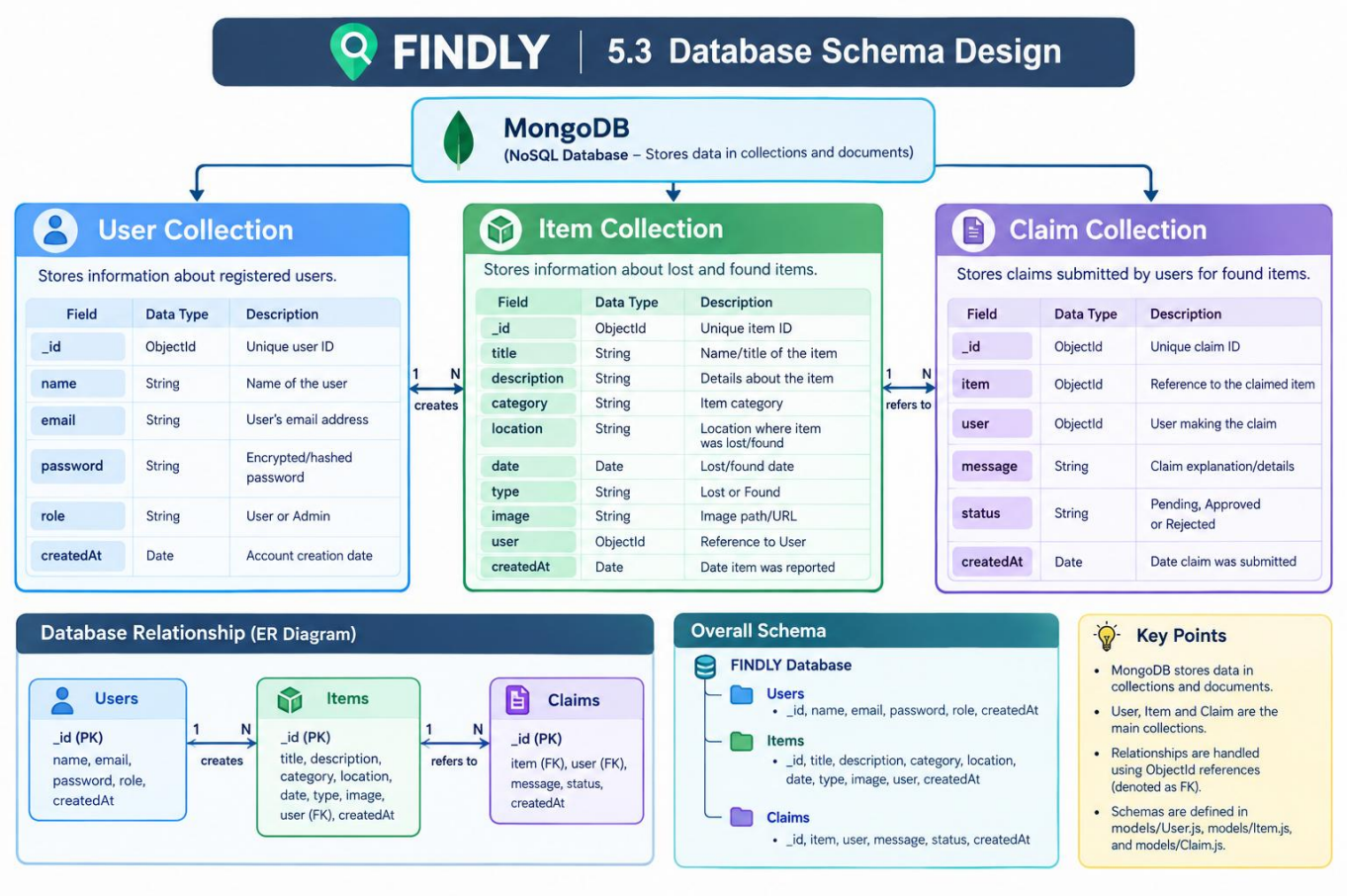
technologies: Node.js/Express, PHP, Python, etc.

5.3 Database Schema Design

The database stores and manages FINDLY's information.

Main Tables

- **Users**
- **Lost_Items**
- **Found_Items**
- **Matches**
- **Claims**
- **Notifications**
- **Admins**



Basic Structure

A database such as **MySQL** can be used to maintain structured and consistent data.

5.4 Website Structure

The **website structure** defines how the FINDLY website is organized and how users navigate between its different sections.

Main Website Sections

1. Home Page

- Introduction to FINDLY

- Search access
- Login/Register options

2. Authentication
- Registration
- Login
- Password management

3. User Dashboard
- My Lost Items
- My Found Items
- My Claims
- Notifications

4. Lost Item Section
- Report Lost Item
- View Lost Items
- Update/Delete Report

5. Found Item Section
- Report Found Item
- View Found Items
- Update/Delete Report

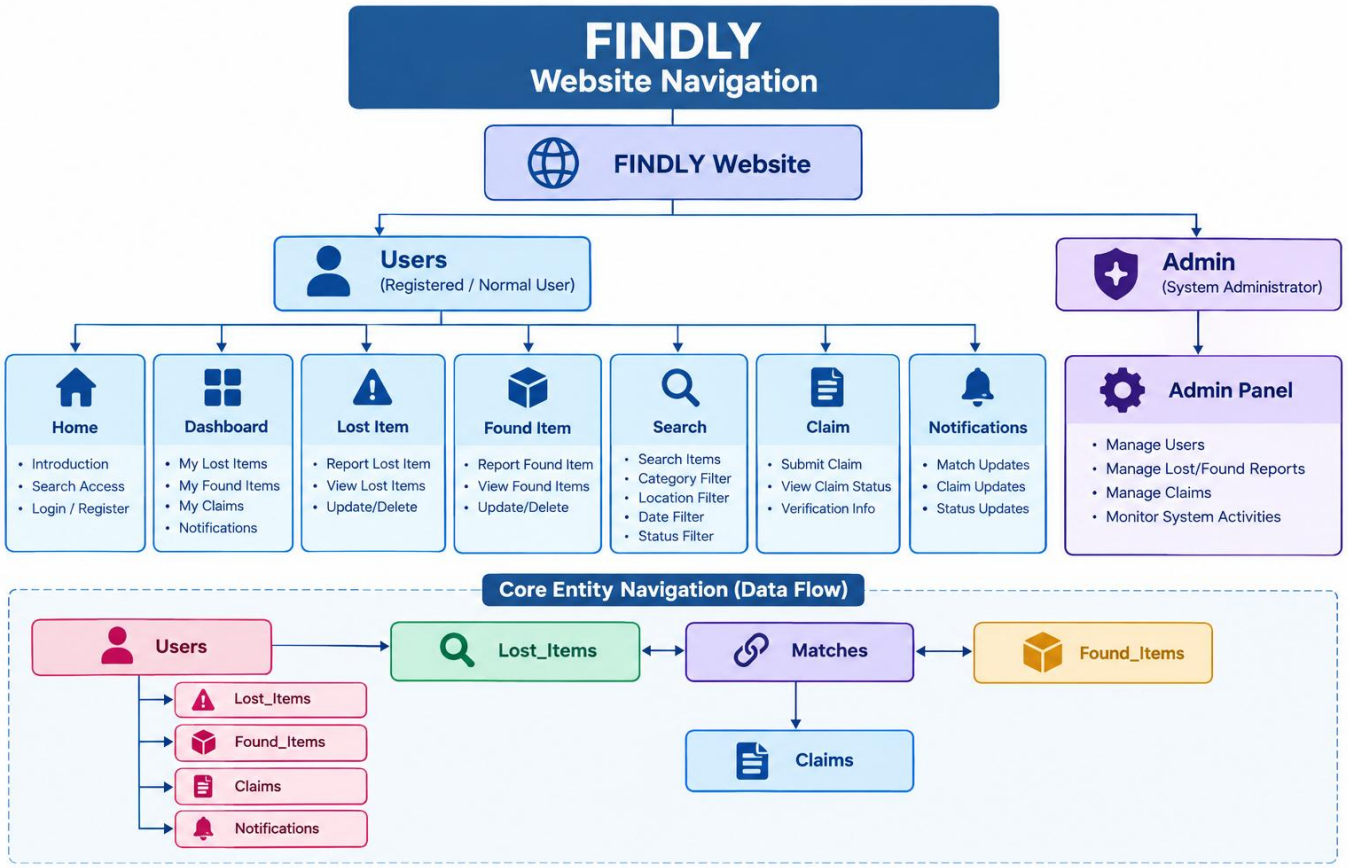
6. Search Section
- Search items
- Category filter
- Location filter
- Date filter
- Status filter

7. Claim Section
- Submit claim
- View claim status
- Verification information

8. Notification Section
- Potential match notifications
- Claim updates
- Status updates

9. Admin Section
- Manage users
- Manage lost/found reports
- Manage claims
- Monitor system activities

Website Navigation



Description

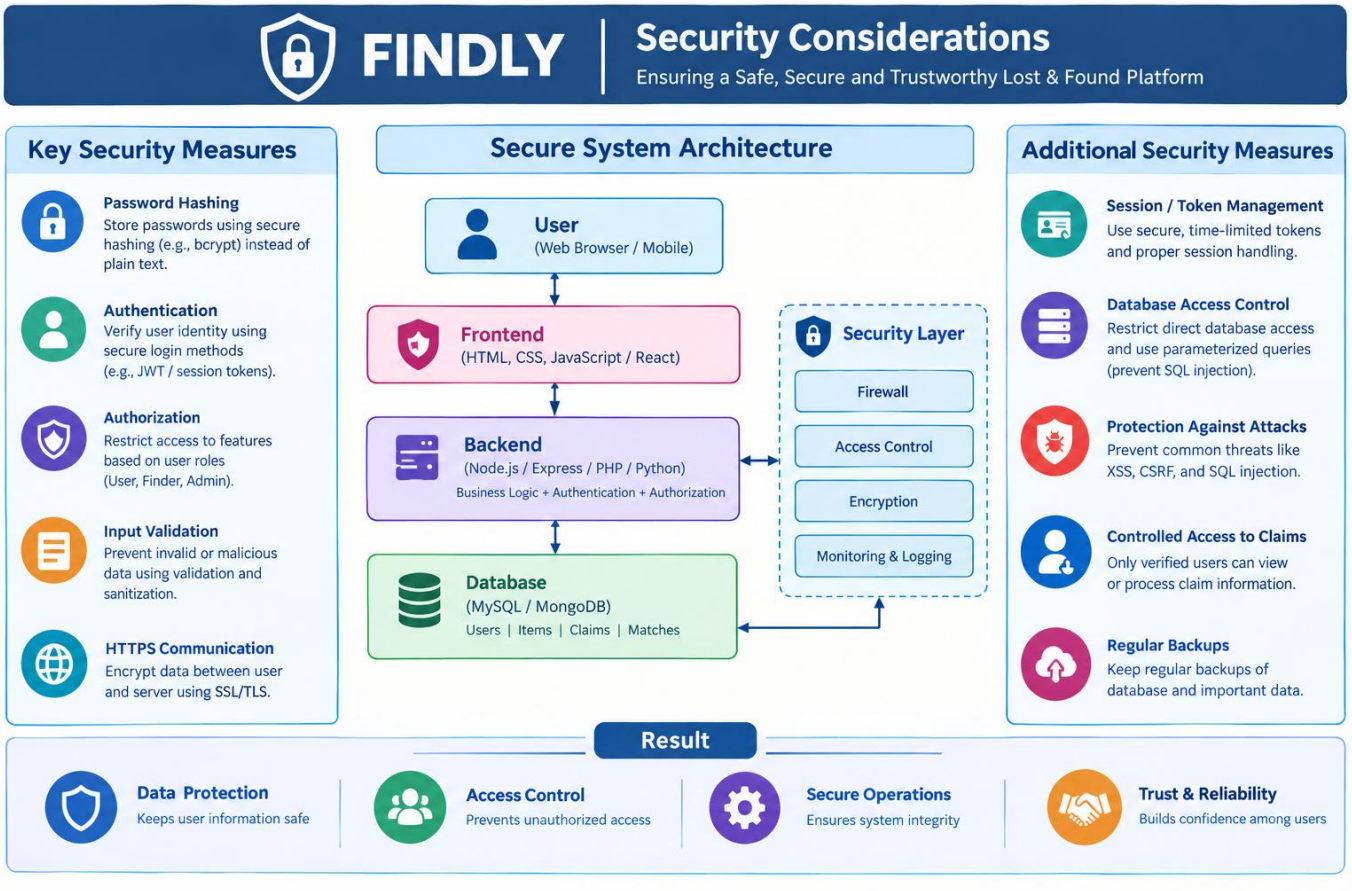
The website structure of FINDLY defines the organization and navigation of the portal. It provides dedicated interfaces for users and administrators, enabling efficient access to lost and found item reporting, searching, item details, claims, notifications, and administrative functions. A structured navigation system improves usability and provides a consistent user experience throughout the application.

5.5 Security Considerations

Security is essential because FINDLY handles user accounts, personal information, and ownership claims.

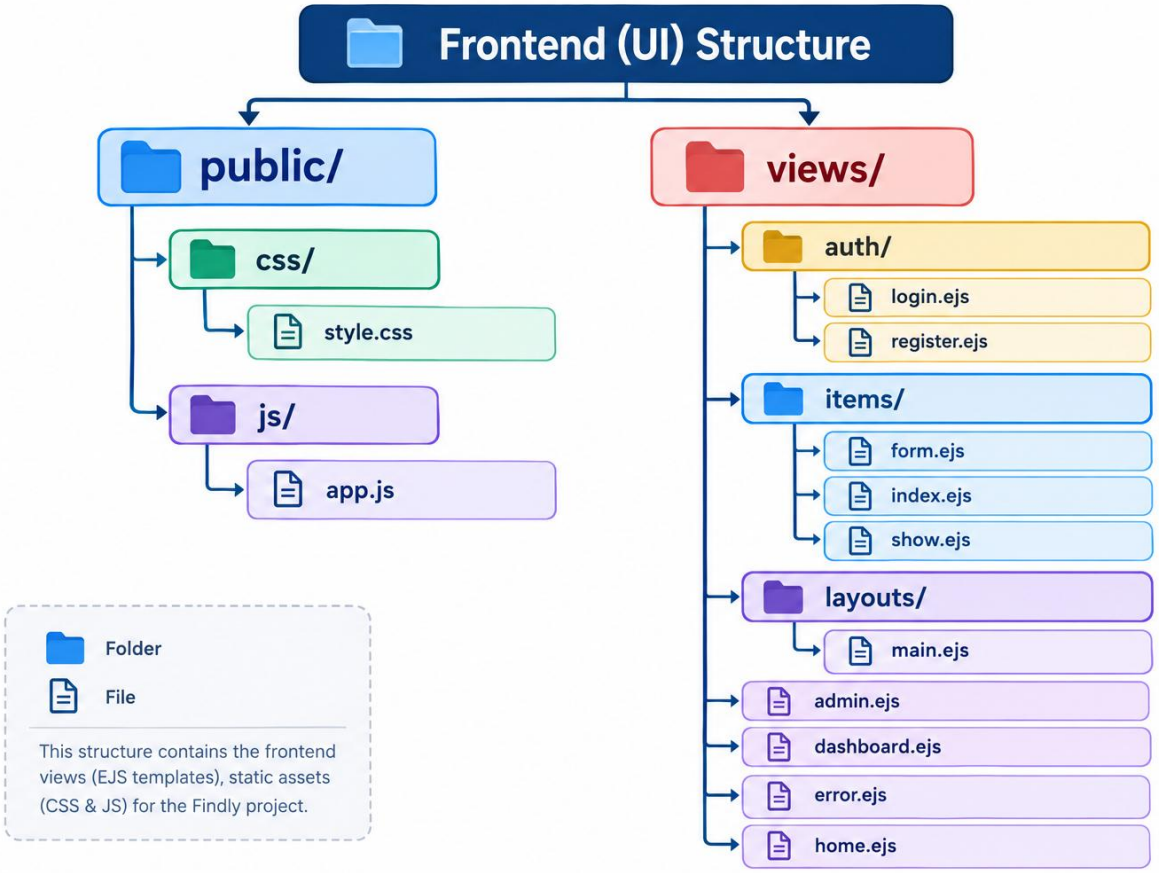
Major Security Measures

- Secure password hashing.
- User authentication.
- Role-based authorization.
- Input validation.
- HTTPS communication.
- Secure session/token management.
- Database access control.
- Protection against unauthorized access.
- Controlled access to ownership claims.
- Regular backup of important data



6. Coding Of Website

6.1 Frontend coding



6.1.1 Login.ejs

```
<section class="auth">
  <p class="eyebrow">Welcome back</p>
  <h1>Sign in to Findly.</h1>
  <form class="form-card" method="POST"><label>Email<input required type="email" name="email"
        placeholder="you@example.com"></label><label>Password<input required type="password"
name="password"
        placeholder="Your password"></label><button class="button wide">Sign in →</button>
    <p class="form-foot">New here? <a href="/register">Create an account</a></p>
  </form>
</section>
```

6.1.2 register.ejs

```
<section class="auth">
  <p class="eyebrow">Join the community</p>
  <h1>Start making returns happen.</h1>
```

```
  <form class="form-card" method="POST"><label>Your name<input required name="name"
          placeholder="Alex Morgan"></label><label>Email<input required type="email" name="email"
          placeholder="you@example.com"></label><label>Password<input required minlength="6"
type="password"
          name="password" placeholder="At least 6 characters"></label><button class="button
wide">Create account
        →</button>
      <p class="form-foot">Already a member? <a href="/login">Sign in</a></p>
  </form>
</section>
```

6.1.3 form.ejs

```
<section class="narrow">
  <p class="eyebrow">
    <%= edit?'Update your listing':'Create a report' %>
  </p>
  <h1>
    <%= edit?'Keep the details current.':'Let's help it find its way.' %>
  </h1>
  <form class="form-card" method="POST" enctype="multipart/form-data"
    action="<%= edit?'/items/'+item._id+'?_method=PUT':'/items' %>"><label>I want to report
a…</label>
    <div class="segmented"><label><input required type="radio" name="type" value="lost"
<%=item.type!=='found'
          ?'checked':'' %>><span>Something I lost</span></label><label><input type="radio"
name="type"
          value="found" <%=item.type==='found' ?'checked':'' %>><span>Something I
found</span></label></div>
    <label>What is it?<input required name="title" value="<%= item.title||'' %>"
        placeholder="e.g. Black leather wallet"></label>
    <div class="two"><label>Category<select required name="category">
          <option value="">Choose one</option>
          <%categories.forEach(c=>{%><option <%=item.category===c?'selected':'' %>><%=c%>
            </option>
            <%})%>
        </select></label><label>Date <input required type="date" name="date"
          value="<%= item.date?new Date(item.date).toISOString().slice(0,10):"
%>"></label></div><label>Where
      was it lost/found?<input required name="location" value="<%= item.location||'' %>"
        placeholder="e.g. Main library, second floor"></label><label>Tell us a little more<textarea
required
        name="description" rows="5"
        placeholder="Include colour, brand, distinguishing features…"><%= item.description||"
%></textarea></label><label>Photo
      <input type="file" name="image" accept="image/*"></label><button class="button wide">
      <%= edit?'Save changes':'Publish report' %> <b>→</b>
    </button>
  </form>
</section>
```

6.1.4 index.ejs

```
<section class="page-title">
  <p class="eyebrow">Community board</p>
  <h1>Find what’s been found.</h1>
  <p>Search the latest reports from your community.</p>
</section>
<form class="filters" method="GET"><input name="q" value="<%= filters.q %>"
    placeholder="Search items, descriptions…"><select name="type">
    <option value="">Lost & found</option>
    <option value="lost" <%=filters.type==='lost'?'selected':''%>>Lost</option>
    <option value="found" <%=filters.type==='found'?'selected':''%>>Found</option>
  </select><select name="category">
    <option value="">All categories</option>
    <% categories.forEach(c=>{%><option <%=filters.category===c?'selected':''%>><%= c %>
      </option>
      <%})%>
  </select><input name="location" value="<%= filters.location %>" placeholder="Location"><button
    class="button">Search</button></form>
<p class="result-count">
  <%= items.length %> listing<%= items.length===1?'':'s' %> found
</p>
<section class="grid listings">
  <% items.forEach(item=>{ %><article class="item-card"><a class="item-image" href="/items/<%=
item._id %>">
        <% if(item.image){%><img src="<%= item.image %>" alt="<%= item.title %>">
          <%}else{%><span>
              <%= item.category==='Keys' ?'🔑':item.category==='Electronics' ?'🔳':'◌' %>
            </span>
            <%}%>
      </a>
      <div class="item-body">
        <div><span class="tag <%= item.type %>">
            <%= item.type %>
          </span><span class="date">
            <%= new Date(item.date).toLocaleDateString() %>
          </span></div>
        <h3><a href="/items/<%= item._id %>">
            <%= item.title %>
          </a></h3>
        <p>⌖ <%= item.location %>
        </p>
      </div>
    </article>
    <%})%>
</section>
<%if(!items.length){%>
  <div class="empty">
    <h2>Nothing matched that search.</h2>
    <p>Try a different phrase, category, or location.</p><a class="button" href="/items">Clear filters</a>
  </div>
```

<%}%>

6.1.5 show.ejs

```
<section class="detail">
  <div class="detail-image">
    <%if(item.image){%><img src="<%=item.image%>" alt="<%=item.title%>">
      <%}else{%><span>
          <%=item.category==='Keys'?'🔑':item.category==='Electronics'?'📱':'📦'%>
        </span>
        <%}%>
  </div>
  <div class="detail-content"><a class="back" href="/items">← Back to listings</a>
    <div class="detail-meta"><span class="tag <%=item.type%>">
        <%=item.type%>
      </span><span class="status <%=item.status%>">
        <%=item.status%>
      </span></div>
    <h1>
      <%=item.title%>
    </h1>
    <p class="location">📍 <%=item.location%> <span>·</span>
        <%=new
Date(item.date).toLocaleDateString(undefined,{month:'long',day:'numeric',year:'numeric'})%>
    </p>
    <p class="description">
      <%=item.description%>
    </p>
    <div class="reporter">Reported by <b>
        <%=item.owner.name%>
      </b> · <%=new Date(item.createdAt).toLocaleDateString()%>
    </div>
    <%if(currentUser && String(item.owner._id)===currentUser._id){%><a class="button"
        href="/items/<%=item._id%>/edit">Edit listing</a>
      <%}else if(currentUser && item.status==='open' ){%><button class="button" data-modal="claim-
modal">This
          might be mine →</button>
        <%}else if(!currentUser){%><a class="button" href="/login">Sign in to claim →</a>
          <%}%>
  </div>
</section>
<%if(claims.length){%>
  <section class="claims">
    <h2>Claims received</h2>
    <%claims.forEach(claim=>{%><article><b>
          <%=claim.claimant.name%>
        </b><span class="status <%=claim.status%>">
          <%=claim.status%>
        </span>
        <p>
          <%=claim.message%>
```

```
            </p><small>
               <%=claim.proof%>
            </small>
         </article>
         <%})%>
   </section>
   <%}%>
      <div class="modal" id="claim-modal">
         <div class="modal-card"><button class="close" data-close>×</button>
            <p class="eyebrow">Verify ownership</p>
            <h2>Tell the reporter why it’s yours.</h2>
            <form method="POST" action="/items/<%=item._id%>/claims"><label>Message<textarea
required name="message"
                     rows="4"
                     placeholder="Describe identifying details only the owner would
know."></textarea></label><label>Optional
                  proof<input name="proof" placeholder="Receipt, serial number, etc."></label><button
                  class="button wide">Submit claim</button></form>
         </div>
      </div>
```

6.1.6 main.ejs

```
<!doctype html>
<html lang="en">

<head>
   <meta charset="utf-8">
   <meta name="viewport" content="width=device-width,initial-scale=1">
   <title>
      <%= typeof title !=='undefined' ? title+'
         · ' : '' %>Findly</title><link rel="preconnect" href="https://fonts.googleapis.com"><link
rel="preconnect" href="https://fonts.gstatic.com" crossorigin><link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Fraunces:opsz,
wght@9..144,600;9..144,700&display=swap" rel="stylesheet"><link rel="stylesheet"
href="/css/style.css"></head><body><header><a class="brand" href="/"><span>f</span>
findly</a><nav><a class="<%= path==='
         /'?'active':'' %>" href="/items">Browse</a><a href="/items/new">Report an item</a>
         <% if(currentUser){ %><a href="/dashboard">My dashboard</a>
            <% if(currentUser.role==='admin' ){ %><a href="/admin">Admin</a>
               <% } %>
                  <form action="/logout" method="POST"><button class="nav-button">Sign
out</button></form><span
                     class="avatar">
                     <%= currentUser.name.charAt(0) %>
                  </span>
                  <% } else { %><a href="/login">Sign in</a><a class="button small" href="/register">Join
                        Findly</a>
                     <% } %>
                        </nav>
                        </header>
```

```
                    <main>
                        <% if(success){ %>
                            <div class="flash success">
                                <%= success %>
                            </div>
                            <% } %>
                                <% if(error){ %>
                                    <div class="flash error">
                                        <%= error %>
                                    </div>
                                    <% } %><%- body %>
                    </main>
                    <footer><a class="brand" href="/"><span>f</span> findly</a>
                        <p>Making lost things easier to find.</p>
                        <p>© <%= new Date().getFullYear() %> Findly</p>
                    </footer>
                    <script src="/js/app.js"></script>
                    </body>

</html>
```

6.1.7 admin.ejs

```
<section class="page-title">
    <p class="eyebrow">Moderation</p>
    <h1>Pending claims</h1>
    <p>Review ownership details before arranging a return.</p>
</section>
<section class="admin-list">
    <%claims.forEach(claim=>{%><article class="claim-card">
            <div><span class="tag <%=claim.item.type%>">
                    <%=claim.item.type%>
                </span>
                <h2><a href="/items/<%=claim.item._id%>">
                        <%=claim.item.title%>
                    </a></h2>
                <p><b>
                        <%=claim.claimant.name%>
                    </b> believes this item is theirs.</p>
                <blockquote>
                    <%=claim.message%>
                </blockquote><small>Proof: <%=claim.proof||'No additional proof supplied'%></small>
            </div>
            <div class="claim-actions">
                <form method="POST" action="/claims/<%=claim._id%>?_method=PUT"><input
type="hidden" name="status"
                        value="approved"><button class="button">Approve</button></form>
                <form method="POST" action="/claims/<%=claim._id%>?_method=PUT"><input
type="hidden" name="status"
                        value="declined"><button class="outline">Decline</button></form>
            </div>
```

```
      </article>
      <%})%>
        <%if(!claims.length){%>
          <div class="empty">
            <h2>You’re all caught up.</h2>
            <p>There are no pending claims to review.</p>
          </div>
          <%}%>
</section>
```

6.1.8 dashboard.ejs

```
<section class="page-title split">
  <div>
    <p class="eyebrow">Your space</p>
    <h1>Welcome back,
      <%=currentUser.name.split(' ')[0]%>.</h1><p>Keep an eye on your reports and
claims.</p></div><a class="button" href="/items/new">+ Report an item</a></section><section
class="dashboard-grid"><div><h2>Your reports</h2><%if(items.length){items.forEach(item=>{%><a
class="row-card" href="/items/<%=item._id%>"><span class="row-icon"><%=item.category==='
          Keys'?'🔑':'◌'%></span><span><b>
            <%=item.title%>
          </b><small>
            <%=item.type%> · <%=item.location%>
          </small></span><span class="status <%=item.status%>">
          <%=item.status%>
        </span><span>→</span></a>
        <%})}else{%>
          <div class="empty compact">
            <p>You have not reported anything yet.</p><a href="/items/new">Create your first
report</a>
          </div>
          <%}%>
  </div>
  <div>
    <h2>Your claims</h2>
    <%if(claims.length){claims.forEach(claim=>{%><a class="row-card"
href="/items/<%=claim.item._id%>"><span
          class="row-icon">⌕</span><span><b>
            <%=claim.item.title%>
          </b><small>Submitted <%=new
Date(claim.createdAt).toLocaleDateString()%></small></span><span
          class="status <%=claim.status%>">
          <%=claim.status%>
        </span></a>
      <%})}else{%>
        <div class="empty compact">
          <p>No claims submitted yet.</p><a href="/items">Browse listings</a>
        </div>
        <%}%>
  </div>
```

```
</section>
```

6.1.9 error.ejs

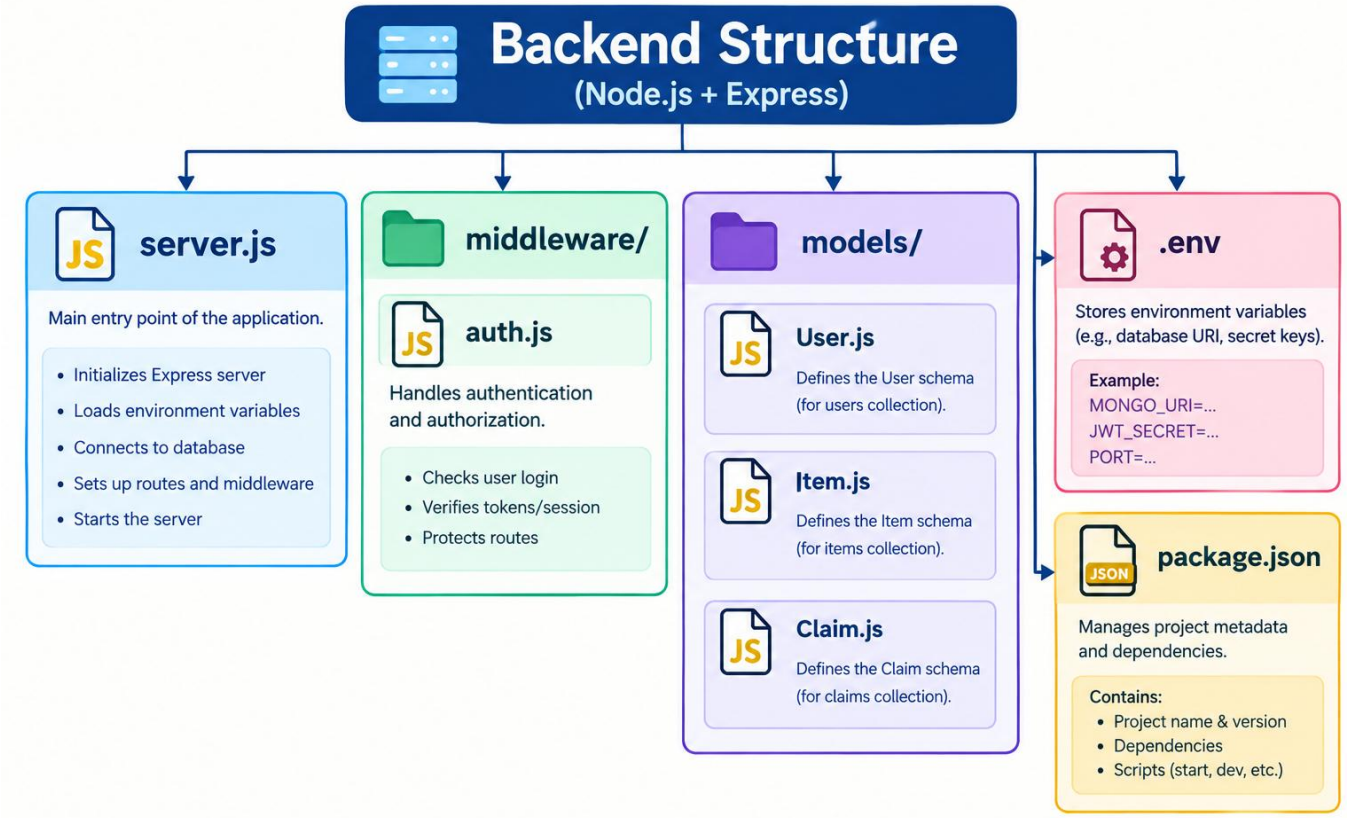
```
<section class="empty error-page">
  <p class="eyebrow">Oops</p>
  <h1>
    <%=message%>
  </h1><a class="button" href="/">Back home</a>
</section>
```

6.1.10 home.ejs

```
<section class="hero">
  <div>
    <p class="eyebrow">Community lost & found</p>
    <h1>Bring lost things<br>back <em>home.</em></h1>
    <p class="lead">A kinder, simpler place for your community to report, search, and reunite with what
matters.</p>
    <div class="hero-actions"><a class="button" href="/items/new">Report an item <b>→</b></a><a
class="text-link"
        href="/items">Browse listings</a></div>
    <div class="trust"><span>✦</span> Free for everyone. Built for good.</div>
  </div>
  <div class="hero-art">
    <div class="sun"></div>
    <div class="blob b1"></div>
    <div class="blob b2"></div>
    <div class="card floating"><span class="tag found">FOUND</span><strong>Silver
keyring</strong><small>Central
        Library · Today</small></div>
    <div class="card lower"><span class="tag lost">LOST</span><strong>Blue water
bottle</strong><small>North Campus
        · Yesterday</small></div>
    <div class="illustration">ρ<div class="spark">✦</div>
    </div>
  </div>
</section>
<section class="stats">
  <div><b>
      <%= stats.lost %>
    </b><span>Items looking<br>for their way back</span></div>
  <div><b>
      <%= stats.found %>
    </b><span>Kind people who<br>found something</span></div>
  <div><b>
      <%= stats.resolved %>
    </b><span>Happy reunions<br>made possible</span></div>
</section>
<section class="section-head">
  <div>
```

```
      <p class="eyebrow">Fresh from the community</p>
      <h2>Recent reports</h2>
   </div><a class="text-link" href="/items">View all listings →</a>
</section>
<section class="grid">
   <% recent.forEach(item=>{ %><article class="item-card"><a class="item-image" href="/items/<%=
item._id %>">
            <% if(item.image){ %><img src="<%= item.image %>" alt="<%= item.title %>">
              <% } else { %><span>
                    <%= item.category==='Keys' ?'🔑':item.category==='Electronics' ?'🔳':'◌' %>
                 </span>
                 <% } %>
         </a>
         <div class="item-body">
            <div><span class="tag <%= item.type %>">
                 <%= item.type %>
              </span><span class="date">
                 <%= new Date(item.date).toLocaleDateString() %>
              </span></div>
            <h3><a href="/items/<%= item._id %>">
                 <%= item.title %>
              </a></h3>
            <p>⌖ <%= item.location %>
            </p>
         </div>
      </article>
      <% }) %>
         <% if(!recent.length){ %>
            <p>No reports yet. Be the first to help!</p>
            <% } %>
</section>
<section class="how">
   <p class="eyebrow">How it works</p>
   <h2>Small actions, big reunions.</h2>
   <div class="steps">
      <div><i>1</i>
         <h3>Report it</h3>
         <p>Share what you lost or found in less than a minute.</p>
      </div>
      <div><i>2</i>
         <h3>Connect safely</h3>
         <p>Browse matches and submit a claim with identifying details.</p>
      </div>
      <div><i>3</i>
         <h3>Make the return</h3>
         <p>Confirm ownership and arrange a safe handover.</p>
      </div>
   </div>
</section>
```

6.2 Backend coding



6.2.1 server.js

```
require("dotenv").config();
const path = require("path"),
  express = require("express"),
  mongoose = require("mongoose"),
  session = require("express-session"),
  MongoStore = require("connect-mongo"),
  layout = require("express-ejs-layouts"),
  methodOverride = require("method-override"),
  multer = require("multer");
const User = require("./models/User"),
  Item = require("./models/Item"),
  Claim = require("./models/Claim");
const { ensureAuth, ensureAdmin } = require("./middleware/auth");
const app = express();
const upload = multer({
  dest: path.join(__dirname, "public/uploads"),
  limits: { fileSize: 4 * 1024 * 1024 },
});
mongoose
  .connect(process.env.MONGODB_URI || "mongodb://127.0.0.1:27017/findly")
```

```
.then(() => console.log("MongoDB connected"))
  .catch((err) => console.error("MongoDB connection error:", err.message));
app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));
app.use(layout);
app.set("layout", "layouts/main");
app.use(express.static(path.join(__dirname, "public")));
app.use(express.urlencoded({ extended: true }));
app.use(methodOverride("_method"));
app.use(
  session({
    secret: process.env.SESSION_SECRET || "change-me",
    resave: false,
    saveUninitialized: false,
    store: MongoStore.create({
      mongoUrl: process.env.MONGODB_URI || "mongodb://127.0.0.1:27017/findly",
    }),
  }),
);
app.use((req, res, next) => {
  res.locals.currentUser = req.session.user;
  res.locals.path = req.path;
  res.locals.success = req.session.success;
  res.locals.error = req.session.error;
  delete req.session.success;
  delete req.session.error;
  next();
});
const categories = [
  "Electronics",
  "Keys",
  "Wallet & ID",
  "Bags",
  "Clothing",
  "Jewelry",
  "Documents",
  "Other",
];
app.get("/", async (req, res) => {
  const recent = await Item.find({ status: "open" })
    .sort("-createdAt")
    .limit(6)
    .populate("owner");
  const [lost, found, resolved] = await Promise.all([
    Item.countDocuments({ type: "lost", status: "open" }),
    Item.countDocuments({ type: "found", status: "open" }),
    Item.countDocuments({ status: "resolved" }),
  ]);
  res.render("home", { recent, stats: { lost, found, resolved } });
});
app.get("/items", async (req, res) => {
```

```
  const { q = "", type = "", category = "", location = "" } = req.query;

  const filter = { status: { $ne: "resolved" } };
  if (q) filter.$text = { $search: q };
  if (type) filter.type = type;
  if (category) filter.category = category;
  if (location) filter.location = new RegExp(location, "i");
  const items = await Item.find(filter).sort("-createdAt").populate("owner");
  res.render("items/index", {
    items,
    categories,
    filters: { q, type, category, location },
  });
});
app.get("/items/new", ensureAuth, (req, res) =>
  res.render("items/form", { item: {}, categories, edit: false }),
);
app.post("/items", ensureAuth, upload.single("image"), async (req, res) => {
  const item = await Item.create({
    ...req.body,
    image: req.file ? "/uploads/" + req.file.filename : "",
    owner: req.session.user._id,
  });
  req.session.success = "Your report is now live.";
  res.redirect("/items/" + item._id);
});
app.get("/items/:id", async (req, res) => {
  const item = await Item.findById(req.params.id).populate("owner");
  if (!item)
    return res.status(404).render("error", { message: "Item not found." });
  const claims =
    req.session.user &&
    (String(item.owner._id) === req.session.user._id ||
      req.session.user.role === "admin")
      ? await Claim.find({ item: item._id }).populate("claimant")
      : [];
  res.render("items/show", { item, claims });
});
app.get("/items/:id/edit", ensureAuth, async (req, res) => {
  const item = await Item.findById(req.params.id);
  if (
    !item ||
    (String(item.owner) !== req.session.user._id &&
      req.session.user.role !== "admin")
  )
    return res
      .status(403)
      .render("error", { message: "You cannot edit this item." });
  res.render("items/form", { item, categories, edit: true });
});
app.put("/items/:id", ensureAuth, upload.single("image"), async (req, res) => {
  const item = await Item.findById(req.params.id);
```

```
if (
    !item ||

  (String(item.owner) !== req.session.user._id &&
     req.session.user.role !== "admin")
  )
    return res.status(403).render("error", { message: "Not allowed." });
  Object.assign(item, req.body);
  if (req.file) item.image = "/uploads/" + req.file.filename;
  await item.save();
  req.session.success = "Listing updated.";
  res.redirect("/items/" + item._id);
});
app.post("/items/:id/claims", ensureAuth, async (req, res) => {
  const item = await Item.findById(req.params.id);
  if (!item || item.status !== "open") {
    req.session.error = "This listing is no longer accepting claims.";
    return res.redirect("/items/" + req.params.id);
  }
  await Claim.create({
    item: item._id,
    claimant: req.session.user._id,
    message: req.body.message,
    proof: req.body.proof,
  });
  req.session.success = "Claim submitted for review.";
  res.redirect("/dashboard");
});
app.get("/dashboard", ensureAuth, async (req, res) => {
  const items = await Item.find({ owner: req.session.user._id }).sort(
    "-createdAt",
  );
  const claims = await Claim.find({ claimant: req.session.user._id })
    .populate("item")
    .sort("-createdAt");
  res.render("dashboard", { items, claims });
});
app.get("/admin", ensureAuth, ensureAdmin, async (req, res) => {
  const claims = await Claim.find({ status: "pending" })
    .populate("item claimant")
    .sort("-createdAt");
  res.render("admin", { claims });
});
app.put("/claims/:id", ensureAuth, ensureAdmin, async (req, res) => {
  const claim = await Claim.findById(req.params.id).populate("item");
  if (!claim) return res.redirect("/admin");
  claim.status = req.body.status;
  if (req.body.status === "approved") {
    claim.item.status = "claimed";
    await claim.item.save();
  }
```

```
  await claim.save();
  req.session.success = "Claim " + req.body.status + ".";
  res.redirect("/admin");
});

app.get("/register", (req, res) => res.render("auth/register"));
app.post("/register", async (req, res) => {
  try {
    const u = await User.create(req.body);
    req.session.user = {
      _id: String(u._id),
      name: u.name,
      email: u.email,
      role: u.role,
    };
    res.redirect("/");
  } catch (e) {
    req.session.error = "An account already exists with that email.";
    res.redirect("/register");
  }
});
app.get("/login", (req, res) => res.render("auth/login"));
app.post("/login", async (req, res) => {
  const u = await User.findOne({ email: req.body.email.toLowerCase() });
  if (!u || !(await u.comparePassword(req.body.password))) {
    req.session.error = "Incorrect email or password.";
    return res.redirect("/login");
  }
  req.session.user = {
    _id: String(u._id),
    name: u.name,
    email: u.email,
    role: u.role,
  };
  res.redirect("/dashboard");
});
app.post("/logout", (req, res) => req.session.destroy(() => res.redirect("/")));
app.use((req, res) =>
  res.status(404).render("error", { message: "That page does not exist." }),
);
app.use((err, req, res, next) => {
  console.error(err);
  res
    .status(500)
    .render("error", { message: "Something went wrong. Please try again." });
});
app.listen(process.env.PORT || 3000, () =>
  console.log(
    "Findly running on http://localhost:" + (process.env.PORT || 3000),
  ),
);
```

6.2.2 auth.js

```
exports.ensureAuth = (req, res, next) => req.session.user ? next() : res.redirect('/login');
exports.ensureAdmin = (req, res, next) => req.session.user?.role === 'admin' ? next() :
res.status(403).render('error', { message: 'Admin access is required.' });
```

6.2.3 claim.js

```
const mongoose = require('mongoose');
module.exports = mongoose.model('Claim', new mongoose.Schema({
  item: { type: mongoose.Schema.Types.ObjectId, ref: 'Item', required: true },
  claimant: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  message: { type: String, required: true }, proof: String,
  status: { type: String, enum: ['pending', 'approved', 'declined'], default: 'pending' }
}, { timestamps: true }));
```

6.2.4 item.js

```
const mongoose = require('mongoose');
const itemSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true }, description: { type: String, required: true },
  type: { type: String, enum: ['lost', 'found'], required: true },
  category: { type: String, required: true }, location: { type: String, required: true },
  date: { type: Date, required: true }, image: String,
  status: { type: String, enum: ['open', 'claimed', 'resolved'], default: 'open' },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
}, { timestamps: true });
itemSchema.index({ title: 'text', description: 'text', location: 'text' });
module.exports = mongoose.model('Item', itemSchema);
```

6.2.5 user.js

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true, minlength: 6 },
  role: { type: String, enum: ['user', 'admin'], default: 'user' }
}, { timestamps: true });
userSchema.pre('save', async function () { if (!this.isModified('password')) return; this.password = await
bcrypt.hash(this.password, 12); });
userSchema.methods.comparePassword = function (password) { return bcrypt.compare(password,
this.password); };
module.exports = mongoose.model('User', userSchema);
```

6.2.6 package.json

```
{
  "name": "findly-lost-found",
  "version": "1.0.0",
  "description": "Lost and found MVP built with Express, MongoDB and EJS",
  "main": "server.js",
  "scripts": {
    "dev": "nodemon server.js",
    "start": "node server.js",
    "seed": "node seed.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "connect-mongo": "^5.1.0",
    "dotenv": "^16.4.5",
    "ejs": "^3.1.10",
    "express": "^4.19.2",
    "express-ejs-layouts": "^2.5.1",
    "express-session": "^1.18.0",
    "method-override": "^3.0.0",
    "mongoose": "^8.5.1",
    "multer": "^1.4.5-lts.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

7. Application Development

This phase converts the system design into a **working FINDLY application**.

7.1 Frontend Implementation

The frontend uses HTML, CSS and JavaScript and provides a responsive interface for users.

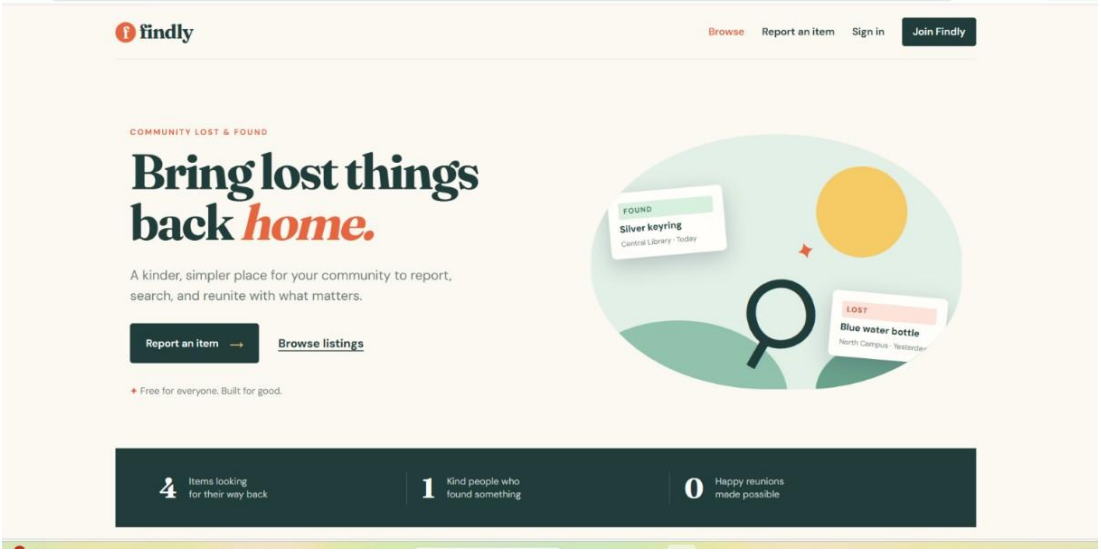


Figure 7.1 – Findly Home Page

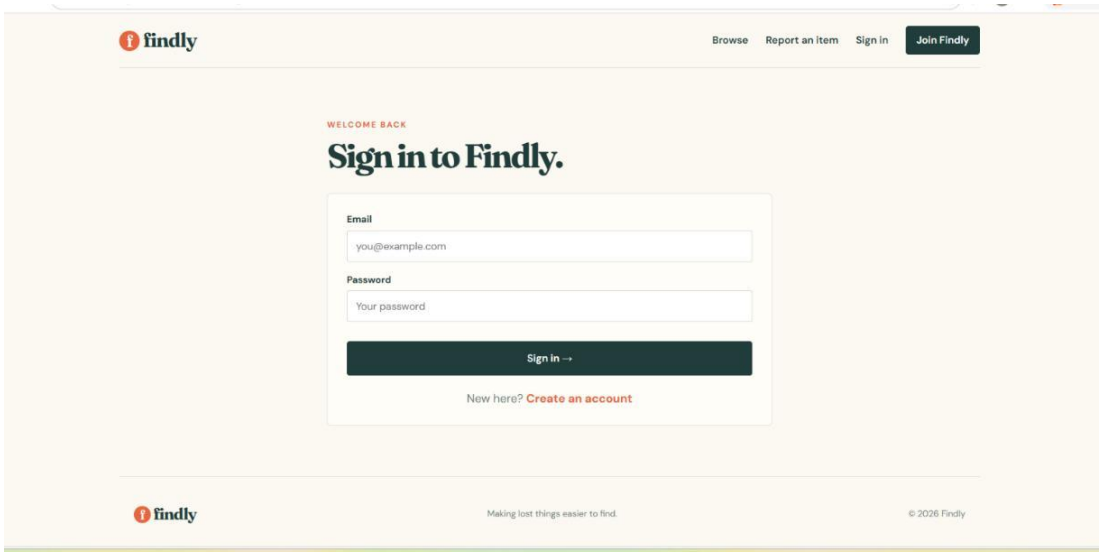


Figure 7.2 – Findly Sign-in Page

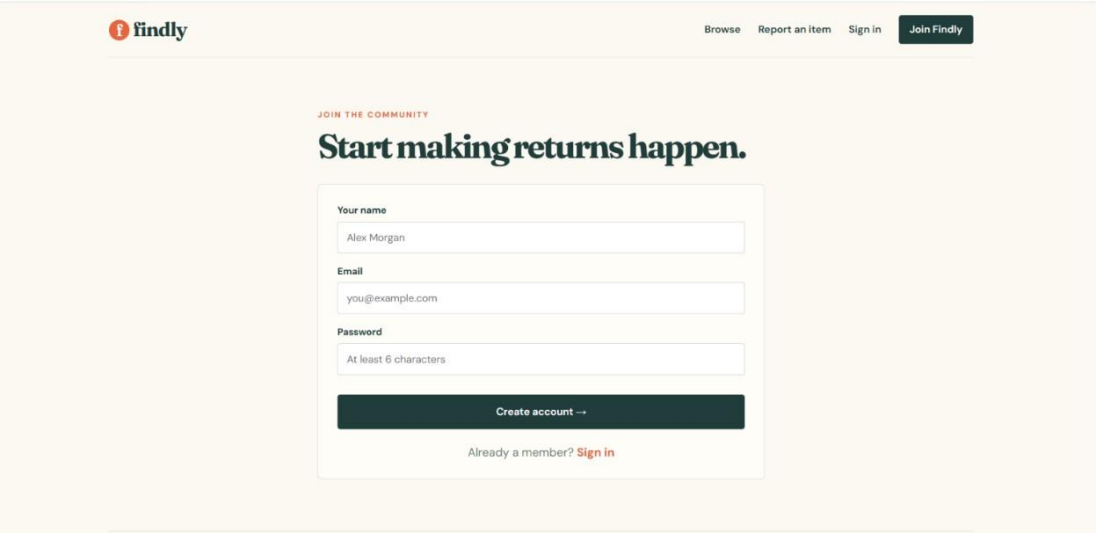


Figure 7.3 – Findly Account Creation Page

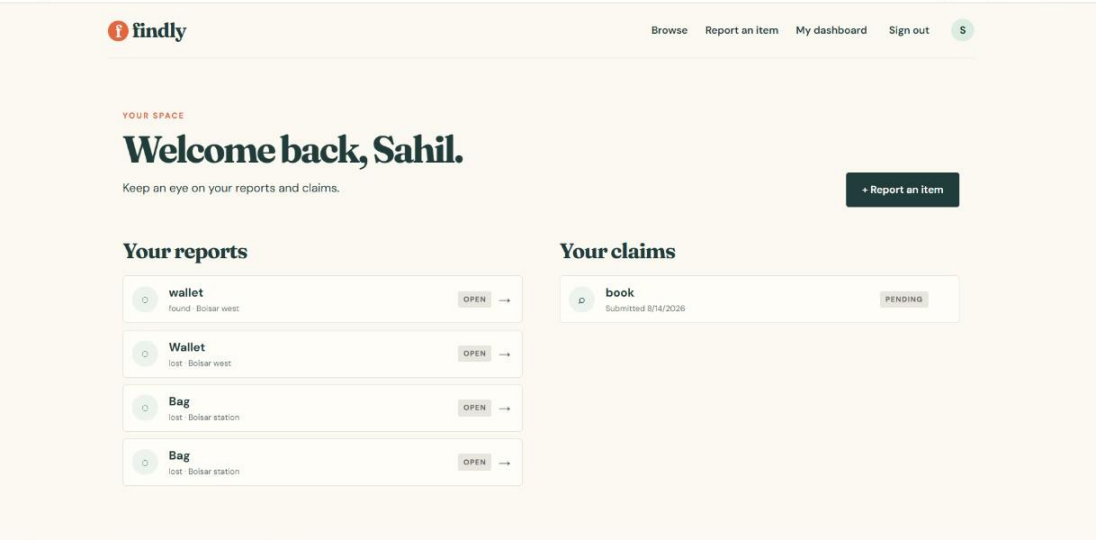


Figure 7.4 – Findly Dashboard

7.2 Backend Implementation

The backend handles the **business logic and processing** of FINDLY.

It manages:

- User authentication
- Lost and found reports
- Search operations
- Item matching
- Claims and verification
- Notifications
- Admin operations

The backend communicates between the frontend and database.

7.3 Database Integration

The database stores all important FINDLY data.

Main tables include:

- Users
- Lost Items
- Found Items
- Matches
- Claims
- Notifications
- Admins

The application performs operations such as **create, read, update, and delete (CRUD)** on the database.

7.4 Authentication and Validation

Authentication ensures that only registered users can access protected features.

The system includes:

- User registration
- Login/logout
- Password protection
- Role-based access
- Form validation
- Unauthorized-access prevention

Input validation ensures that users submit **complete and valid information**.

7.5 Error Handling

Error handling ensures that the system responds properly when something goes wrong.
Examples:

- Invalid login → Display an error message.
- Missing item information → Ask the user to complete required fields.
- Database failure → Display a suitable system message.
- Unauthorized access → Deny access to the protected page.

8. Integration & System Testing

Testing ensures that FINDLY works correctly and meets its requirements.

8.1 Unit Testing

Individual components are tested separately.

Examples:

- Login function
- Registration function
- Lost-item form
- Search function
- Claim function

8.2 Black-Box Testing

The system is tested based on **inputs and expected outputs** without examining the internal code.

Example:
Input: Search for "Black Wallet"
Expected Output: Relevant wallet reports are displayed.

8.3 Integration Testing

Integration testing checks whether different modules work correctly together.

Example:
Lost Item Report
↓
Database
↓
Matching Module
↓
Notification

This ensures that information flows correctly between modules.

8.4 Test Case Preparation

Test cases define what should be tested and what result is expected.

Test Case	Input	Expected Result
Login	Valid credentials	Dashboard displayed
Login	Invalid credentials	Error message
Lost Item	Valid details	Report saved
Search	Item keyword	Matching results
Claim	Valid claim	Claim submitted
Admin Login	Valid admin details	Admin dashboard

8.5 Bug Tracking

Identified problems are recorded, analyzed, and corrected.

Typical bug information includes:

- Bug ID
- Description
- Module
- Severity
- Status
- Resolution

9. Application Deployment

Deployment makes FINDLY available for actual use.

9.1 Cloud Deployment

The existing project documentation identifies Render as the cloud deployment platform. The application can be accessed through a web browser after deployment.

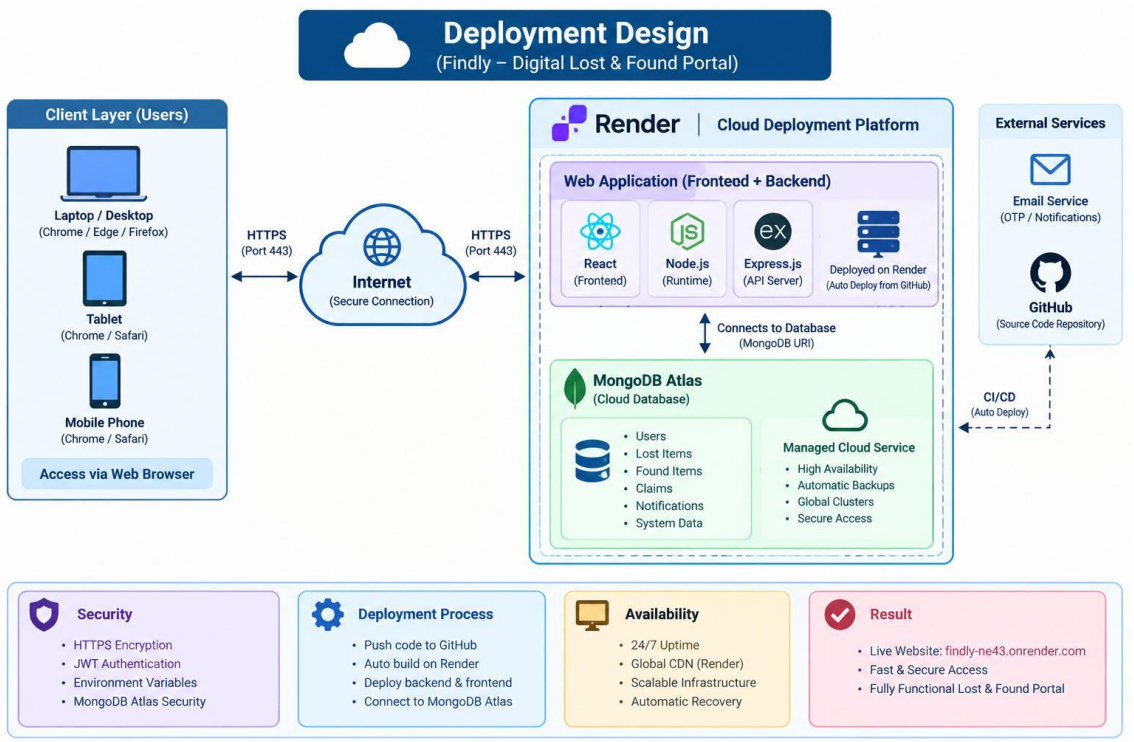


Figure 9.1 – Deployment Design

9.2 Local Hosting

During development and testing, FINDLY can be run locally using a development server.

Example:
User → Browser → Local Server → Database

9.3 APK Build

If FINDLY is developed as a mobile application, an **Android APK** can be generated for installation and testing.

If FINDLY is only a web application, this section can be marked **Not Applicable**.

9.4 Server Configuration

Server configuration includes:

- Database connection
- Environment variables

- Domain configuration
- HTTPS/SSL
- Application settings
- Server permissions

9.5 Version Control Using GitHub

GitHub can be used to:

- Store source code
- Track changes
- Maintain different versions
- Collaborate with team members
- Manage project releases

10. Performance & Security Testing

This phase evaluates the **speed, reliability, and security** of FINDLY.

10.1 Basic Load Testing

Load testing checks how the system performs when multiple users access it simultaneously.

Examples:

- Multiple users searching items.
- Multiple reports being submitted.
- Several users accessing the dashboard.

10.2 Input Validation Checks

The system checks whether user input is valid.

Examples:

- Required fields cannot be empty.
- Email format must be valid.
- Invalid characters should be handled appropriately.
- Uploaded files should meet permitted requirements.

10.3 Security Validation

Security testing checks whether unauthorized users can access protected information or functions.

It includes:

- Authentication testing
- Authorization testing
- Password protection
- Session security
- Database access control
- Input security
- Protection of personal information

11. Final Documentation

The final documentation records the complete development and operation of FINDLY.

11.1 Technical Report

The technical report contains:

- Introduction
- Problem statement
- Requirements
- Feasibility study
- System design
- UML diagrams
- Implementation
- Testing
- Deployment
- Results
- Future scope

11.2 User Manual

The user manual explains how to use FINDLY.

For example:
Register
↓
Login
↓
Report Lost / Found Item
↓
Search for Matches
↓
Submit Claim
↓
Verification
↓
Item Returned

11.3 Application Screenshots

Screenshots demonstrate the actual working application, such as:

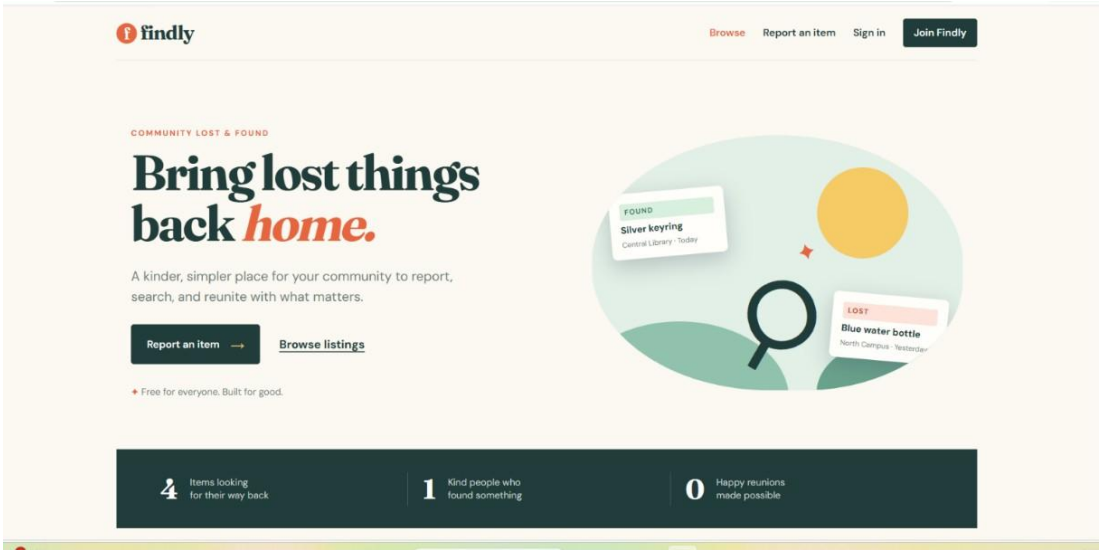


Figure 11.1 – Findly Home Page

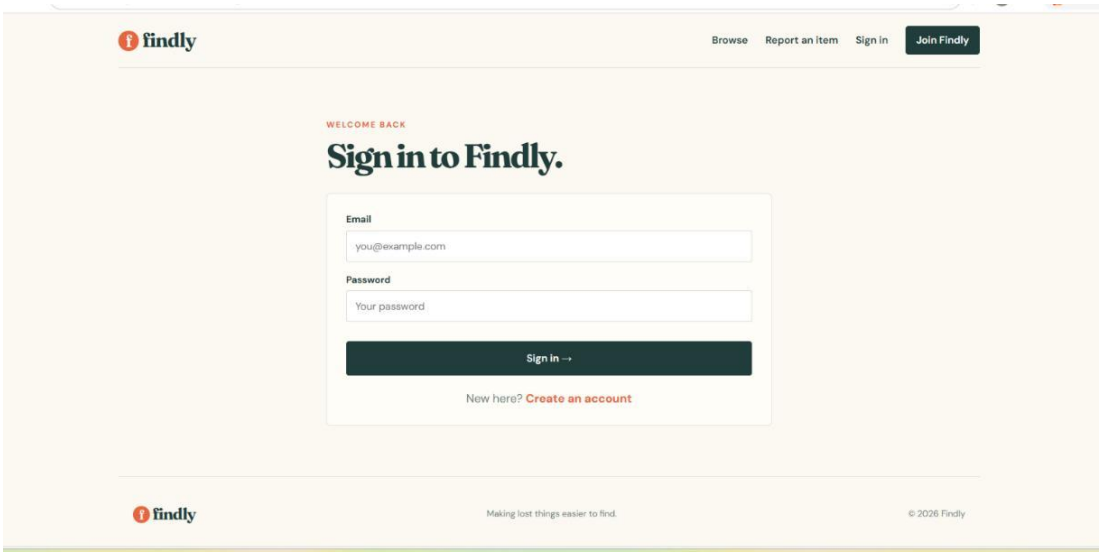


Figure 11.2 – Findly Sign-in Page

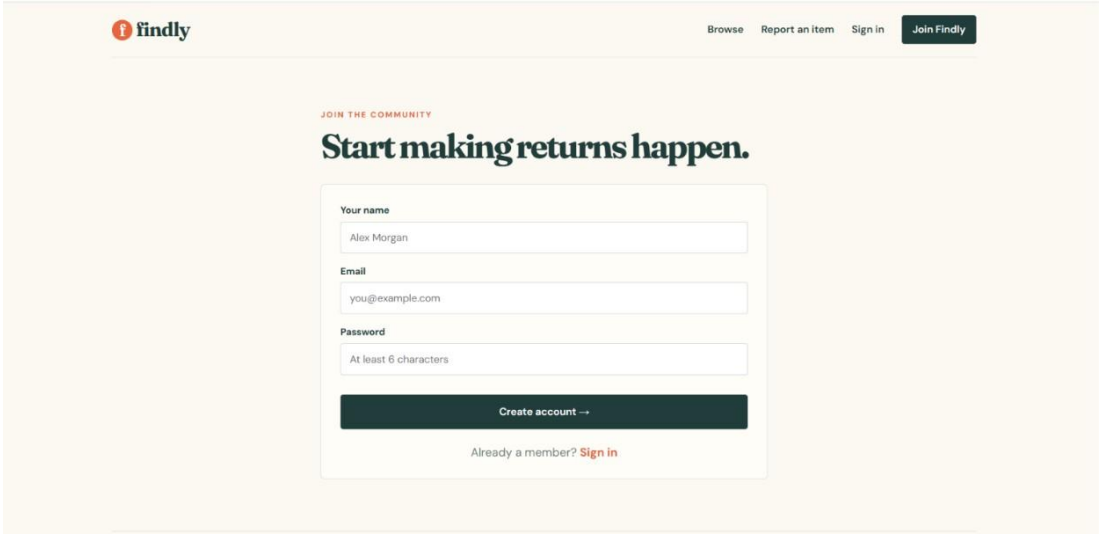


Figure 11.3 – Findly Account Creation Page

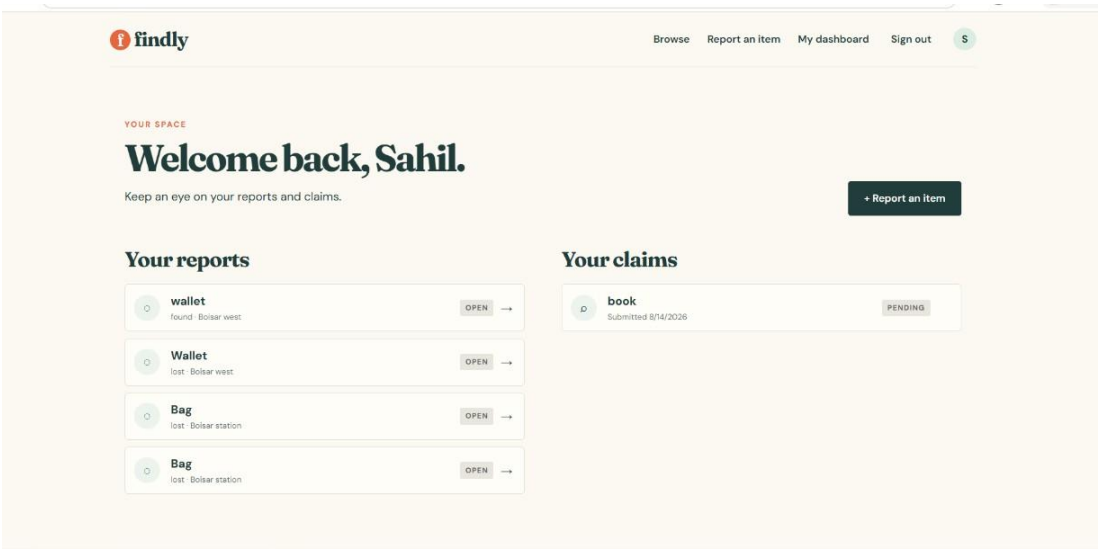


Figure 11.4 – Findly Dashboard

11.4 Source Code Documentation

This section explains the organization and purpose of the source code, including:

- Frontend files
- Backend files
- Database files
- Configuration
- Important functions
- Installation instructions

12. Reference

URL:- https://findly-ne43.onrender.com/dashboard

1. MDN Web Docs. HTML, CSS and JavaScript Documentation. https://developer.mozilla.org/
2. Node.js. Official Node.js Documentation. https://nodejs.org/docs/
3. Express.js. Express Web Framework Documentation. https://expressjs.com/
4. MongoDB. MongoDB Database Documentation. https://www.mongodb.com/docs/
5. MongoDB Atlas. Cloud Database Documentation. https://www.mongodb.com/atlas
6. GitHub. GitHub Documentation. https://docs.github.com/
7. Render. Web Application Deployment Documentation. https://render.com/docs
8. W3Schools. HTML, CSS and JavaScript Tutorials. https://www.w3schools.com/
9. GeeksforGeeks. Web Development Tutorials and Programming Resources. https://www.geeksforgeeks.org/
10. Stack Overflow. Programming Questions and Answers. https://stackoverflow.com/
11. Community Interactions. Observations and information collected during the community visit for the Digital Lost and Found for Public Spaces project. 2026.

13.Glossary

Term	Definition
FINDLY	A web-based Community Lost & Found Portal used to report, search, match, and claim lost or found items.
Authentication	Process of verifying a user's identity during login.
Authorization	Process of determining what actions a user is allowed to perform.
Admin	Authorized person responsible for managing users, reports, claims, and system activities.
Frontend	The visible part of the website through which users interact with FINDLY.
Backend	The server-side part of the application that processes requests and manages business logic.
Database	Organized collection of data used to store users, items, claims, and other system information.
Lost Item	An item reported by a user as missing.
Found Item	An item reported by a user as being found.
Claim	A request submitted by a user to prove ownership of a found item.
Matching	Process of comparing lost and found item information to identify possible matches.
Notification	Message sent to users about matches, claims, or other important system activities.
UML	Unified Modeling Language, used to visually represent the structure and behavior of a software system.
Use Case	A description of an interaction between a user and the system to achieve a specific goal.
ER Diagram	Entity Relationship Diagram showing entities, attributes, and relationships in a database.
API	Application Programming Interface used for communication between different software components.
CRUD	Create, Read, Update, and Delete operations performed on database records.
SDLC	Software Development Life Cycle, a structured process for developing and maintaining software.
Agile	An iterative software development approach that emphasizes continuous development, testing, and feedback.
Input Validation	Process of checking whether user-provided data is valid and safe.
Password Hashing	Converting a password into a secure irreversible representation before storing it.
Session	Temporary information used to maintain a user's logged-in state.
HTTPS	Secure version of HTTP that encrypts communication between the browser and server.
Deployment	Process of making the application available for users on a server or hosting platform.
GitHub	Platform used for storing, managing, and version-controlling source code.
Load Testing	Testing the system under multiple simultaneous users or requests to evaluate performance.

Bug	An error or defect in the software that causes incorrect or unexpected behavior.
Test Case	A defined set of inputs, conditions, and expected results used to test a system feature.
Responsive Design	Design approach that allows the website to work properly on different screen sizes and devices.
Role-Based Access Control (RBAC)	Security mechanism that provides system permissions according to a user's role.

14. Appendices – FINDLY

The appendices contain additional information and supporting materials used during the development, testing, and evaluation of the **FINDLY – Community Lost & Found Portal**.

Appendix A – Survey Questionnaire

A questionnaire was prepared to understand the problems faced by students and community members when they lose or find personal belongings.

Sample Questions:

1. Have you ever lost an item such as a mobile phone, wallet, ID card, or bag?
2. Have you ever found an item belonging to someone else?
3. How do you currently search for a lost item?
4. What problems do you face while searching for lost items?
5. Would you use an online Lost & Found platform?
6. Which features would you find useful?
7. Is ownership verification important before returning an item?
8. Would notifications about potential matches be useful?
9. How important is privacy and security?
10. What improvements would you suggest for FINDLY?

Appendix B – Survey Data Sheet

The survey data sheet contains the responses collected from users.

Sr. No.	Question	Yes	No	Maybe
1	Have you lost an item?	___	___	___
2	Have you found an item?	___	___	___
3	Would you use FINDLY?	___	___	___
4	Is verification important?	___	___	___
5	Are notifications useful?	___	___	___

Actual values can be entered according to the survey conducted.

Appendix C – Sample Test Data

Sample records were created to test the FINDLY application.

Item ID	Item	Category	Location	Status
L001	Mobile Phone	Electronics	Library	Lost
L002	Wallet	Personal	Canteen	Lost
F001	Mobile Phone	Electronics	Library	Found
F002	ID Card	Documents	Classroom	Found

Appendix D – Test Cases

Test cases were prepared to verify the major functions of the application.

Test Case	Expected Result
Valid Login	User enters dashboard
Invalid Login	Error message displayed
Report Lost Item	Item saved successfully
Report Found Item	Found item saved

Search Item	Relevant results displayed
Submit Claim	Claim submitted successfully
Admin Login	Admin dashboard displayed

Appendix E – User Feedback

Users were asked to provide feedback after using the application.

The feedback focused on:

- Ease of registration and login
- Ease of reporting lost/found items
- Search functionality
- User interface
- Claim process
- Notifications
- Suggestions for improvement